

ОТЧЁТ ОБ АУДИТЕ КОДА

RaveClone

SyncWatch / Плинка

Полный аудит безопасности и качества кода
iOS · Backend · React Native

155

всего багов

27

критических

46

высоких

82

средн+низких

Подготовлено для передачи в GLM-5.2 (Z Client)
Источник: github.com/PubgmHacker/xpkcakpkfewp-ofewk-pkv

2026-07-03 · America/Los_Angeles

00 **Краткое резюме**

Проведён полный аудит кодовой базы RaveClone / SyncWatch / Плинк — приложения для совместного просмотра видео в реальном времени. Аудит охватывает три платформы: нативный iOS-клиент (Swift/SwiftUI), Node.js бэкенд (Fastify + WebSocket + Prisma) и React Native-клиент (Expo). Суммарно обнаружено 155 багов, из них 27 критических, блокирующих релиз в текущем виде.

Главная находка аудита: несколько showstopper-багов делают ключевые фичи приложения неработоспособными «из коробки». На iOS не работает весь realtime-слой (WebSocket не переходит в состояние connected), DM/друзья/админка получают 401, host identity захардкожен как «current_user». На бэкенде 12 хендлеров читают несуществующее поле request.user.sub вместо request.user.id, ломая DM/друзей/профиль/историю. На React Native JWT хранится в небезопасном AsyncStorage, а DRM-плеер использует неправильный URL.

Распределение по серьёзности и платформам

Платформа	🔴 Критич.	🟠 Выс.	🟡 Сред.	🟢 Низк.	Итого
Backend (Node.js)	5	11	10	6	34
iOS (Swift/SwiftUI)	14	14	16	16	60
React Native (Expo)	8	21	17	15	61
Итого	27	46	43	37	155

01 **Топ-5 приоритетов (что фиксить первым делом)**

Эти пять исправлений открывают путь к работающему MVP. Без них остальной фикс-ап бессмысленен — приложение либо не запускается в ключевых сценариях, либо не проходит App Store Review.

- 1

iOS C1: починить lifecycle WebSocketClient.isConnected

Что: Вызвать notifyConnectedIfNeeded() в receive-loop после task.resume(), либо открыть сокет через withCheckedThrowingContinuation.

Почему: Без этого ВСЬ realtime-слой (чат, синс, реакции, сигналинг, WebRTC) мёртв. send() всегда копит в pendingMessages и никогда не флешит.
- 2

Backend #1+#2: request.user.sub → .id + preHandler authenticate на admin

Что: Заменить request.user.sub на request.user.id в 12 хендлерах (messages, friends, profile). Добавить preHandler: [fastify.authenticate] на все admin-роуты.

Почему: Сейчас DM, друзья, профиль, история просмотра, премиум и вся админка возвращают 500 или 401. Четыре крупных фичи полностью не работают.

3**iOS C4+C5+C6: внедрить общий authenticated APIClient**

Что: В RaveCloneApp создать один APIClient с инжектированным authToken, передавать его в DMChatService, FriendManager, AdminPanelView через DI.

Почему: Сейчас каждый из этих сервисов создаёт собственный APIClient() без токена. Все их вызовы возвращают 401.

4**iOS C7+C8: прокинуть реальный currentUserId и hostIsPremium**

Что: В RoomView.setupViewModel и RoomCreationView.createRoom использовать authService.currentUser?.id и PremiumStatusManager.shared.isPremium вместо хардкода «current_user»/false.

Почему: isHost всегда false → play/pause/seek не отправляются, sync-движок не запускается. hostIsPremium=false → реклама показывается всем, даже премиум-юзерам.

5**RN C2+C3+C4+C5: SecureStore для JWT и DRM-куки + удалить demo-bypass**

Что: Перевести JWT из AsyncStorage в expo-secure-store. Перенести туда же куки DRM-сессий Netflix/Кинопоиск. Удалить offline-guest fallback и demo Google/VK exchange.

Почему: Текущая модель auth ненадёжна: токены утекают через filesystem, demo-токен «demo_google_token» тривиально логинит любого. App Store отклонит.

02

Спец-задача: не работает анимированный фон в HomeView

Пользователь сообщил: «не работает фон в приложении (анимированный) бэкграунд, который за карточками видео «смотрят сейчас» и «рекомендации»». После анализа `AnimatedGradientBackground.swift`, `HomeView.swift` и `Color+Theme.swift` найдено 5 независимых причин, ни одна из которых не связана с самими карточками. Карточки рендерятся нормально — фон под ними теряется.

Причина #1 (главная): SwiftUI-антипаттерн `withAnimation + repeatForever` в `.onAppear`

▢ RaveClone/RaveClone/Views/Components/AnimatedGradientBackground.swift:53-57

```
.onAppear {
    withAnimation(.easeInOut(duration: 14).repeatForever(autoreverses: true)) {
        animate = true
    }
}
```

Что не так: Известный SwiftUI-антипаттерн. Когда `animate = false` изначально и вы переключаете его на `true` внутри `withAnimation` в `.onAppear`:

1. `.onAppear` срабатывает ДО того, как `view` полностью в иерархии — `withAnimation` запускается, но первый кадр может не зафиксироваться.
2. При возврате из `RoomView` (`navigation back`) `.onAppear` срабатывает снова, но `animate` уже `true` — состояние не меняется, новая анимация не запускается, старая продолжается с непредсказуемым состоянием.
3. `repeatForever(autoreverses: true)` интерполирует смещения `orbs` на основе `boolean`. Между `false` и `true` нет промежуточных кадров — интерполяция может «залипнуть» на первом кадре.

Влияние: Орбы либо статичны (вижу только чёрный фон), либо случайно начинают двигаться после навигации.

Исправление: Использовать `TimelineView` или `Canvas` с `TimelineScheduler` — это гарантирует непрерывную анимацию независимо от `lifecycle`. Альтернатива — отложенный старт через `DispatchQueue.main.asyncAfter`:

```
.onAppear {
    animate = false
    DispatchQueue.main.asyncAfter(deadline: .now() + 0.1) {
        withAnimation(.easeInOut(duration: 14).repeatForever(autoreverses: true)) {
            animate = true
        }
    }
}

// ИЛИ (рекомендуется) — TimelineView с постоянной анимацией:
TimelineView(.animation) { context in
    let t = context.date.timeIntervalSinceReferenceDate
    GlowOrb(color: warmColors[0], size: 360, blur: 90,
            x: CGFloat(sin(t * 0.1) * 60),
            y: CGFloat(cos(t * 0.07) * 180),
            opacity: 0.22)
}
```

Причина #2: Color.raveBackground = чистый чёрный + opacity 0.10-0.22 — слишком тускло

□ Color+Theme.swift:32 + AnimatedGradientBackground.swift:34,41,48

```
static let raveBackground = Color(hex: 0x000000) // чистый чёрный
GlowOrb(color: warmColors[0], size: 360, blur: 90,
  x: animate ? -60 : -110, y: animate ? -180 : -120,
  opacity: hasActiveRooms ? 0.22 : 0.16) // макс 22%
GlowOrb(... opacity: hasActiveRooms ? 0.18 : 0.13)
GlowOrb(... opacity: hasActiveRooms ? 0.15 : 0.10) // мин 10%
```

Что не так: При opacity 0.10-0.22 поверх чистого чёрного #000000 и blur radius 90-110pt орбы практически невидимы глазу, особенно: при внешней освещённости, на устройствах с антигlare-покрытием, в условиях яркого окружения. Даже если анимация работает — её просто не видно.

Исправление: Поднять opacity до 0.35-0.55, либо сделать базовый фон не чисто чёрным, а #050810 (есть в теме raveBgGradient), либо использовать blendMode(.screen) для орбов на чёрном:

```
Circle()
  .fill(color.opacity(0.45)) // ← было 0.22
  .frame(width: size, height: size)
  .blur(radius: blur)
  .blendMode(.screen) // ← добавит свечение
  .offset(x: x, y: y)
  .compositingGroup() // обязательно для blendMode
```

Причина #3: Reduce Motion accessibility не проверяется

Что не так: Если у пользователя в iOS Settings → Accessibility → Motion включён Reduce Motion (≈10-15% юзеров), SwiftUI полностью отключает withAnimation. Орбы остаются в начальной позиции (animate = false) — т.е. смещены на -110, -180 (вне видимой зоны). Код не проверяет UIAccessibility.isReduceMotionEnabled.

Исправление: Для Reduce Motion либо показывать статичные орбы по центру, либо использовать TimelineView (он не зависит от motion-настроек):

```
@Environment(\.accessibilityReduceMotion) var reduceMotion
var body: some View {
  ZStack {
    Color.raveBackground.ignoresSafeArea()
    if reduceMotion {
      // статичные орбы по центру
      ZStack { /* fixed positions */ }
    } else {
      TimelineView(.animation) { ctx in /* animated orbs */ }
    }
  }
}
```

Причина #4: .blur(radius: 90-110) на 360pt circle — тяжеленный GPU-эффект

Что не так: .blur(radius: 90) на 360pt circle — экстремально тяжёлая операция для GPU. iOS может деградировать blur до r=20-30 при нехватке ресурсов, либо вообще не рендерить его на старых устройствах (iPhone X и старше). Симулятор в Xcode может показывать blur, а

на реальном устройстве — нет.

Исправление: Уменьшить blur до 40-60pt (компенсировать повышением opacity), либо использовать RadialGradient вместо blur — это нативный GPU-примитив, в разы быстрее:

```
RadialGradient(
    colors: [color.opacity(0.55), color.opacity(0.0)],
    center: .center,
    startRadius: 0,
    endRadius: 180
)
.frame(width: 360, height: 360)
.offset(x: x, y: y)
// никакого .blur() — RadialGradient сам по себе glow
```

Причина #5: конкуренция .onAppear в HomeView и AnimatedGradientBackground

▢ HomeView.swift:162-167 и AnimatedGradientBackground.swift:53-57

```
// HomeView.swift:162
.onAppear {
    withAnimation(.spring(response: 0.6, dampingFraction: 0.8)) {
        appeared = true
    }
    startCTACollapseTimer()
}
// AnimatedGradientBackground.swift:53 (внутри HomeView)
.onAppear {
    withAnimation(.easeInOut(duration: 14).repeatForever(autoreverses: true)) {
        animate = true
    }
}
```

Что не так: Оба onAppear срабатывают одновременно. SwiftUI может объединить их в один runloop tick — withAnimation для AnimatedGradientBackground может «проиграть» спринговую анимацию appeared, особенно repeatForever часть, потому что SwiftUI видит каскад анимаций и может дропнуть ту, что длиннее.

Исправление: Разнести во времени или использовать .task вместо .onAppear для фоновой анимации:

```
.task {
    try? await Task.sleep(nanoseconds: 300_000_000) // 300мс
    withAnimation(.easeInOut(duration: 14).repeatForever(autoreverses: true)) {
        animate = true
    }
}
```

Готовый drop-in фикс AnimatedGradientBackground.swift

Минимальное изменение, которое починит анимацию фона в большинстве случаев — заменить весь body на версию с TimelineView:

```
struct AnimatedGradientBackground: View {
    var hasActiveRooms: Bool = true
    var body: some View {
        ZStack {
            Color.raveBackground.ignoresSafeArea()

```

```

TimelineView(.animation) { context in
    let t = context.date.timeIntervalSinceReferenceDate
    ZStack {
        glowOrb(color: warmColors[0], t: t, size: 360, blur: 50,
            xAmp: 60, yAmp: 180, xPhase: 0.0, yPhase: 0.3, opacity: 0.40)
        glowOrb(color: warmColors[1], t: t, size: 300, blur: 60,
            xAmp: 100, yAmp: 160, xPhase: 1.2, yPhase: 0.7, opacity: 0.32)
        glowOrb(color: warmColors[2], t: t, size: 240, blur: 70,
            xAmp: 80, yAmp: 40, xPhase: 2.4, yPhase: 1.1, opacity: 0.28)
    }
    .ignoresSafeArea()
}
}
}
private func glowOrb(color: Color, t: Double, size: CGFloat, blur: CGFloat,
    xAmp: CGFloat, yAmp: CGFloat, xPhase: Double, yPhase: Double,
    opacity: Double) -> some View {
    Circle()
        .fill(color.opacity(opacity))
        .frame(width: size, height: size)
        .blur(radius: blur)
        .offset(
            x: CGFloat(sin(t * 0.1 + xPhase)) * xAmp,
            y: CGFloat(cos(t * 0.07 + yPhase)) * yAmp
        )
        .blendMode(.screen)
}
}
}

```

Что изменено:

- TimelineView(.animation) вместо withAnimation + repeatForever — непрерывная анимация, не зависит от lifecycle.
- Opacity поднята с 0.10-0.22 до 0.28-0.40 — орбы теперь видны.
- Blur уменьшен с 90-110 до 50-70pt — компенсация GPU-нагрузки.
- Добавлен .blendMode(.screen) — свечение поверх чёрного вместо затемнения.
- Синусоидальные смещения вместо boolean-toggle — плавное движение без «застреваний».

03 iOS (Swift/SwiftUI) — 60 багов

Аудит 50+ Swift-файлов: networking (WebSocket, REST, signaling), services (sync, voice, auth, media, DM, screen capture, audio, StoreKit, friends, premium, ads), view models, views, models, utilities, resources.

❑ CRITICAL (14) — showstoppers / security / auth-bypass

CRIT**C1**

WebSocket isConnected никогда не становится true — весь realtime мёртв

❑ Networking/WebSocketClient.swift:408-424

Что не так: Метод `notifyConnectedIfNeeded()` объявлен, но нигде не вызывается (grep-verified). Единственное место, где `isConnected` устанавливается в true.

Влияние: Метод `send()` проверяет `if isConnected` (всегда false) → каждое исходящее сообщение копится в `pendingMessages` и НИКОГДА не отправляется. `delegate?.websocketDidConnect` не вызывается, sync engine не запускается. Чат, реакции, play/pause/seek, сигналинг — ничего не доходит до сервера.

Исправление: Использовать неявное открытое состояние `URLSessionWebSocketTask`: сразу после `task.resume()` вызвать `receiveMessage()`, либо открыть сокет через `withCheckedThrowingContinuation`, который `resume`-ится на первом receive callback.

CRIT**C2**

JWT хранится в UserDefaults вместо Keychain

❑ Services/AuthService.swift:130-131

Что не так: `defaults.set(token, forKey: Keys.authToken)`. Также `Keys.savedUser` хранит полный JSON пользователя в UserDefaults.

Влияние: JWT в UserDefaults читается любым, у кого есть доступ к filesystem: sandbox escape, извлечение iCloud-бэкапа, джейлбрейк. Угон аккаунта → полный account takeover. Apple App Review отклонит за нарушение Secure Coding.

Исправление: Использовать существующий `KeychainHelper` (уже определён в `YandexAuthService.swift:238`). Кстати, Yandex-путь уже хранит JWT в Keychain — несогласованность.

CRIT**C3**

getFreshToken() никогда не обновляет токен

❑ Services/AuthService.swift:118-124

Что не так: Обе ветки `if` возвращают одно и то же `authToken`. Нет вызова `/api/auth/refresh`.

Влияние: После истечения 24-часового JWT приложение молча продолжает слать истёкший токен. Сервер возвращает 401, юзер логается без предупреждения. `bridgeAuthToken()` в `RaveCloneApp.init` затем распространяет истёкший токен в `WebSocketClient` и `MediaService`.

Исправление: Реализовать `refreshJWT()`: POST на `/auth/refresh` с refresh-токеном из Keychain, обновить `authToken` и `expiry`.

CRIT C4 DMChatService создаёт собственный APIClient без токена

▢ Services/DMChatService.swift:16

Что не так: `private let api = APIClient()` — свежий клиент на каждый инстанс. `authToken` всегда `nil` и никем не назначается.

Влияние: Фича DM полностью неработоспособна. `loadHistory(friendId:)` молча `return`-ит из `guard api.authToken != nil; sendMessage` получает 401. UI показывает оптимистичные сообщения, но они никогда не доходят до сервера.

Исправление: Инжектировать общий `APIClient` (тот же, что `RaveCloneApp.init` даёт `AuthService`) в `DMChatService` через конструктор.

CRIT C5 FriendManager создаёт собственный APIClient без токена

▢ Services/FriendManager.swift:26

Что не так: Тот же паттерн, что C4 — `private let api = APIClient()`.

Влияние: Весь социальный слой (друзья, реквесты, поиск) сломан — каждый API-вызов возвращает 401. `loadAll()` в `init()` молча `no-op` из-за `guard`.

Исправление: Инжектировать общий `authenticated APIClient`.

CRIT C6 AdminPanelView создаёт собственный APIClient без токена

▢ Views/Admin/AdminPanelView.swift:8

Что не так: Тот же паттерн. `loadUsers` вызывает `api.request("admin/users")`, который всегда 401.

Влияние: Админ-панель не может загрузить или изменить данные. View бесконечно показывает `empty state`; `ban/unban/delete`-действия падают.

Исправление: Инжектировать общий `authenticated APIClient` из `app-контейнера`.

CRIT C7 RoomView.setupViewModel() хардкодит `currentUserId: "current_user"`

▢ Views/Room/RoomView.swift:470-493

Что не так: `userId: "current_user"` и `isHost: room.hostID == "current_user"` — всегда `false` для реальных юзеров с `UUID`.

Влияние: `RoomViewModel.isHost` всегда `false` → `SyncEngine.play()/pause()/seek()` все `guard isHost` и молча `no-op`. Таймер `broadcast` состояния не запускается. Команды `sync` от хоста не отправляются. Восстановление сохранённой позиции также `gated` на `isHost`.

Исправление: Получать реальный `user id` из `AuthService.currentUser?.id` и передавать вниз. Весь `setupViewModel` должен быть удалён в пользу DI из `RaveCloneApp`.

CRIT**C8****RoomCreationView.createRoom()** хардкодит `hostID` и `hostIsPremium: false`

□ Views/Home/RoomCreationView.swift:469-483

Что не так: `hostID: "current_user"`, `hostIsPremium: false` — хардкод. Также в `CreateRoomView.swift:325-337`.

Влияние: Даже если реальный юзер premium, `hostIsPremium=false` отправляется участникам → `AdSessionManager` трактует хоста как non-premium и показывает рекламу. `hostID="current_user"` значит, что проверки `isHost` в других местах никогда не матчат.

Исправление: Использовать `authService.currentUser?.id ?? UUID().uuidString` и `PremiumStatusManager.shared.isPremium`.

CRIT**C9****PremiumStatusManager.setPremium(_:)** позволяет ручную активацию premium без IAP

□ Services/PremiumStatusManager.swift:62-72

Что не так: `setPremium(true)` ставит `isPremium = true` и `subscriptionExpiry = now + 30d`. `isPremium` персистится в `UserDefaults` (строка 105).

Влияние: Любой вызов (или юзер с filesystem-доступом) может переключить флаг `rave_user_is_premium` и получить premium-фичи навсегда — обход StoreKit, обход серверной валидации, обход ad-gating, поднятие caps с 4 до 50 участников. Гарантированный App Review reject (Guideline 3.1.1).

Исправление: Удалить `setPremium`. Premium должен активироваться только через `StoreManager.handleSuccessfulPurchase` И валидироваться серверно: POST StoreKit JWS на `/api/iap/verify`, сервер ставит `isPremium` на User-записи.

CRIT**C10****AdSessionManager.triggerAd()** скипает premium-host check

□ Services/AdSessionManager.swift:122-136

Что не так: Метод `triggerAd()` не вызывает `shouldPlayAd(hostIsPremium:)` (строка 109) — это dead code, который никто не вызывает.

Влияние: Даже premium-хосты видят рекламу — это убивает всё «Premium = no ads» value proposition. В комбинации с C8 (`hostIsPremium` всегда false) КАЖДЫЙ хост видит рекламу.

Исправление: Добавить guard в начало `triggerAd`: `guard shouldPlayAd(hostIsPremium: PremiumStatusManager.shared.isPremium) else { startAdTimer(); return }`.

CRIT **C11** DirectMessage.isOwnMessage проверяет неверный sentinel

▢ Models/DirectMessage.swift:19-21

Что не так: senderID == "current_user", но DMChatService.sendMessage (строка 69-82) ставит senderID = me, где me = currentUserId ?? "me". Реальный UUID, не «current_user».

Влияние: В DMChatView ваши собственные сообщения рендерятся СЛЕВА (как от собеседника) с placeholder-аватаркой. Premium shimmer (строка 219) никогда не триггерится для own messages.

Исправление: Прокинуть реальный currentUserId в DMChatView и сравнивать там. Или хранить currentUserId в модели при конструировании.

CRIT **C12** Info.plist без privacy usage descriptions (микрофон, камера, галерея, local network)

▢ Resources/Info.plist

Что не так: Нет NSMicrophoneUsageDescription, NSCameraUsageDescription, NSPhotoLibraryUsageDescription, NSLocalNetworkUsageDescription. Приложение имеет voice chat, screen capture, avatar upload — всем нужны эти ключи.

Влияние: AVAudioSession.requestRecordPermission и AVCaptureSession.startRunning крашнут приложение с EXC_BREAKPOINT при первом вызове. App Store авто-реджект.

Исправление: Добавить 4 ключа в Info.plist с русскими описаниями.

CRIT **C13** Plink.entitlements пустой — нет Associated Domains, IAP, APNs

▢ Resources/Plink.entitlements

Что не так: Файл — пустой <dict/>. Нет com.apple.developer.associated-domains, com.apple.developer.in-app-payments, aps-environment.

Влияние: Universal Links, IAP и push-нотификации молча не работают в release-сборках. Deep-link share flow возвращает 404 по тапу. Premium-подписки могут инициироваться, но не завершаются серверной верификацией.

Исправление: Добавить entitlements: applinks:raveclone.app, merchant.com.syncwatch.raveclone, aps-environment: production.

CRIT **C14** Yandex OAuth clientID — захардкоженный placeholder

▢ Services/YandexAuthService.swift:40-41

Что не так: clientID: String = "yandex_client_id_placeholder". Любой код, инстанцирующий YandexAuthService() без override, попадёт на Yandex OAuth с несуществующим client_id.

Влияние: Yandex ID sign-in не работает. Yandex Plus подписка (isPlus) никогда не проверяется.

Исправление: Перенести реальный clientID в build configuration / xcconfig, читать через Bundle.main.object(forKey: "YANDEX_CLIENT_ID").

❏ HIGH (14) — leaks / crashes / lifecycle

HIGH

H1

WebSocketClient.scheduleReconnect может оставить isReconnecting застрявшим

❏ Networking/WebSocketClient.swift:390-401

Что не так: Если disconnect() вызывается во время backoff-окна, isManuallyDisconnected становится true, asyncAfter-замыкание выходит, но isReconnecting остаётся true (сбрасывается только в success-ветке).

Влияние: После backgrounding + manual disconnect клиент может отказаться переподключаться.

Исправление: Сбросить isReconnecting = false в disconnect().

HIGH

H2

RoomViewModel и RoomSyncManager дерутся за wsClient.delegate

❏ Views/Room/RoomView.swift:489 + Services/RoomSyncManager.swift:101

Что не так: Оба объекта назначают себя wsClient.delegate = self. Последний выигрывает. setupViewModel (через .onAppear) идёт первым → RoomSyncManager становится делегатом, затем joinRoomFlow (в .task) перезаписывает на RoomViewModel.

Влияние: Race condition: входящие WS-сообщения могут идти не в тот handler. Два делегата с расходящейся логикой маршрутизации.

Исправление: Использовать один делегат (вероятно RoomViewModel) и явно форвардить в RoomSyncManager, либо multicast-delegate паттерн.

HIGH

H3

VideoContainerView создаёт ВТОРОЙ AVPlayer отдельно от SyncEngine

❏ Views/Room/VideoContainerView.swift:114-146

Что не так: PlayerUIView создаёт собственный AVPlayer из того же URL. SyncEngine.loadMedia ТОЖЕ создаёт AVPlayer. Два AVPlayer играют один поток независимо.

Влияние: SyncEngine контролирует скрытый AVPlayer; юзер видит другой AVPlayer (рассинхронизированный). Каждый play/pause/seek нужно реэплать через updateUIView с 1.5s tolerance, который конфликтует с time observer sync engine. Визуальная десинхронизация гарантирована. CPU/memory тратится на дублирующий decode.

Исправление: Рендерить AVPlayer из SyncEngine через единый AVPlayerLayer, экспортированный из SyncEngine.

HIGH

H4

AdSessionManager.deinit не инвалидейт таймеры

▢ Services/AdSessionManager.swift:66-68

Что не так: Комментарий в коде врёт: `Timer.scheduledTimer retains target/closure` через `run loop`. `deinit` только пишет «timers invalidate themselves».

Влияние: Экземпляры `AdSessionManager` текут, пока `stopAllTimers()` не вызван явно. Если комната закрыта без `stopAllTimers`, таймер продолжает стрелять в мёртвой комнате, вызывая фантомные ad-триггеры.

Исправление: `nonisolated deinit { adTimer?.invalidate(); countdownTimer?.invalidate() }.`

HIGH

H5

AdPlayerView.startCountdown Timer не чистится на dismiss

▢ Views/Room/AdPlayerView.swift:73-88

Что не так: `Timer.scheduledTimer` создаётся в `startCountdown()`, инвалидируется только когда `countdown ≤ 0`. Если view закрыт раньше 15s, таймер продолжает стрелять.

Влияние: После раннего dismiss `onDismiss` вызывается ~15 раз за следующие 15s, каждый раз потенциально триггеря `AdSessionManager.finishAd()` и рестартуя ad-цикл.

Исправление: Хранить `Timer` в `@State`, инвалидировать в `.onDisappear`.

HIGH

H6

AudioManager.animateVolume спавнит 10 racing Tasks

▢ Services/AudioManager.swift:103-122

Что не так: `for step in 1...steps { Task { @MainActor in ... player.volume = ... } }` — 10 detached Tasks на каждый вызов.

Влияние: Если `animateVolume` вызывается дважды за 300ms (`mute→unmute`), 20 task-ов перекрываются, каждый пишет `player.volume` с устаревшим delta. Финальная громкость непредсказуема. CPU просыпается 10x за 300ms.

Исправление: Отменять предыдущий animation Task перед новым: `volumeAnimTask?.cancel(); volumeAnimTask = Task { ... }.`

HIGH

H7

AmbilightSampler.processFrame передаёт CVPixelBuffer в Task.detached без retain

▢ Views/Room/AmbilightBackground.swift:79-84

Что не так: `Task.detached { ... extractDominantColors(from: pixelBuffer, ...) }`. Swift bridges `CVPixelBuffer` как `unretained`. Если source `AVPlayerItemVideoOutput` перерабатывает буфер до выполнения task, читается freed memory.

Влияние: Use-after-free. Возможны интермиттентные краши в `extractDominantColors` под memory pressure или на медленных устройствах.

Исправление: Ручной retain: `CVPixelBufferRetain(pixelBuffer); defer { CVPixelBufferRelease(pixelBuffer) }.`

HIGH

H8

PlayerUIView CADisplayLink retains self forever

□ Views/Room/VideoContainerView.swift:157-193

Что не так: CADisplayLink(target: self, ...) retains target. deinit { displayLink?.invalidate() } работает, но deinit не вызывается, если parent-view остаётся жив, а URL меняется — старый PlayerUIView остаётся с активным display link.

Влияние: Display link течёт между сменами URL; каждая новая загрузка media добавляет ещё один 4Hz capture callback. После 10 смен media — 10 display links вызывают AmbilightSampler.shared.processFrame per frame.

Исправление: Override willMove(toSuperview:): при newSuperview == nil инвалидировать display link, поставить player на pause.

HIGH

H9

RoomView.setupViewModel создаёт свежие сервисы на каждый appear

□ Views/Room/RoomView.swift:469-511

Что не так: Каждый раз при показе RoomView: новый APIClient() (без токена), новый WebSocketClient() (без JWT), новый AuthService, SignalingClient, VoiceChatService, SyncEngine, RoomViewModel, RoomSyncManager.

Влияние: Каждый вход в комнату стартует неаутентифицированную сессию. Сохранённый JWT в AuthService (с app launch) никогда не пробрасывается. Создание комнаты, joining, sync — все 401. При dismiss все сервисы деаллоцируются, но замыкания manager.connect() capture-ят vm strongly, удерживая граф до WebSocket timeout.

Исправление: Инжектировать сервисы из app-контейнера через EnvironmentValue или конструктор.

HIGH

H10

APIClient.encoder/decoder — mutable shared state на Sendable-классе

□ Networking/APIClient.swift:5-18

Что не так: APIClient объявлен Sendable, но JSONEncoder/Decoder хранятся как let. Эти классы НЕ thread-safe (конфигурируют внутреннее состояние при encode/decode). Concurrent request(...) из разных Task-ов разделяют экземпляры.

Влияние: Случайные decode-ошибки, повреждённые request bodies, или краши (EXC_BAD_ACCESS) под нагрузкой. Воспроизводится сложно.

Исправление: Сделать APIClient actor, либо создавать encoder/decoder на каждый request.

HIGH **H11** APIClient.request<T> не обрабатывает 204 No Content

□ Networking/APIClient.swift:62-64

Что не так: case 200..**<300**: return try decoder.decode(T.self, from: data). Многие REST-эндпоинты возвращают 204 с пустым body. JSONDecoder.decode бросает на пустом Data.

Влияние: requestNoBody существует как workaround, но любой, кто случайно использует request<EmptyResponse> против 204-эндпоинта, получает confusing decode-ошибку.

Исправление: Если data.isEmpty и T — EmptyDecodable, вернуть default.

HIGH **H12** WebSocketClient nonisolated(unsafe) var socket — accessed from multiple isolation contexts

□ Networking/WebSocketClient.swift:53

Что не так: private nonisolated(unsafe) var socket: URLSessionWebSocketTask? Accessed from: connectInternal (@MainActor), disconnect (@MainActor), cancelSocketForDeinit (nonisolated), sendRaw (background queue).

Влияние: Data race на socket?.cancel() и socket = nil, если reconnect срабатывает во время deinit с background queue. Краши возможны под TSan.

Исправление: Обернуть в lock: private let socketLock = NSLock().

HIGH **H13** RoomViewModel.messages не ограничен — длинные комнаты текут memory

□ ViewModels/RoomViewModel.swift:13 + Views/Room/RoomView.swift:158-167

Что не так: var messages: [ChatMessage] = []. Каждое входящее сообщение append-ится. Нет upper bound. RoomSyncManager имеет maxChatMessages = 200, но RoomViewModel поддерживает свой массив.

Влияние: В 6-часовой комнате с активным чатом массив растёт до десятков тысяч ChatMessage (по ~200 байт каждый) → 10+ MB мёртвой истории. SwiftUI LazyVStack ре-рендерится на каждый append.

Исправление: Cap на 200 как в RoomSyncManager: if messages.count > maxMessages { messages.removeFirst(...) }.

HIGH **H14** AuthService.init мутирует currentUser из detached Task

□ Services/AuthService.swift:32-35

Что не так: Task { @MainActor in self.currentUser = user }. AuthService не @MainActor, только currentUser аннотирован. RaveCloneApp.checkAuth сразу после init вызывает authService.currentUser(), racing с этим detached task.

Влияние: Cold launch: currentUser() возвращает nil, потому что restore-task ещё не выполнен. isSignedIn = false, splash исчезает на LoginView, через 200ms currentUser появляется — юзер видит флэш login-экрана на каждом cold launch с валидной сессией.

Исправление: Сделать AuthService полностью @MainActor, либо restore синхронно в init через Mutex/actor.

📁 MEDIUM (16) — race / state / @MainActor violations

MED

M1

SyncEngine.handleSeek extrapolation может реверсировать реальные seek

📁 Services/SyncEngine.swift:389-417

Что не так: `isStatePulse = elapsedSinceEvent < stateBroadcastInterval + 1` — эвристика. Реальный seek и периодический state broadcast могут оба попасть в 3s окно.

Влияние: Реальные seek от хоста молча реверсируются. UI показывает seek, затем snap-back.

Исправление: Добавить distinct command value для state pulses vs explicit seeks, либо поле `isStatePulse: Bool` в `SyncMessage`.

MED

M2

SignalingMessage.decode использует `raw.contains` для определения типа

📁 Networking/SignalingMessage.swift:43-47

Что не так: `guard raw.contains("\\"kind\\")`. Любое chat-сообщение, содержащее литерал "kind", пройдёт guard и будет попытано как `SignalingMessage decode`.

Влияние: Occasional swallowed chat messages. Хрупко.

Исправление: Использовать dedicated type field для routing, либо decode через `JSONDecoder` с проверкой kind после.

MED

M3

RoomViewModel.routeInbound декодирует каждое сообщение до 4 раз

📁 ViewModels/RoomViewModel.swift:217-250

Что не так: Последовательные `try? JSONDecoder().decode(...)` для `SyncMessage`, `ChatMessage`, `ParticipantUpdate`.

Влияние: CPU waste на hot path; risk mis-routing, если поля overlap-ят.

Исправление: Peek single type field once: `JSONSerialization.jsonObject` + switch.

MED

M4

RoomSyncManager.handleRawMessage — тот же паттерн multi-decode

📁 Services/RoomSyncManager.swift:195-221

Что не так: Sequence: `WSPingPong` → `AdCommandPayload` → `SyncMessage` → `RoomEventEnvelope`.

Влияние: Тот же, что M3.

Исправление: Тот же fix — единый type-dispatch.

MED **M5** StoreManager.restorePurchases — фактически no-op

□ Services/StoreManager.swift:115-126

Что не так: AppStore.sync() только ре-синхронит StoreKit transaction cache. НЕ итерирует Transaction.currentEntitlements для проверки активных подписок.

Влияние: Юзеры, переустанавливающие приложение на новом устройстве, жмут «Restore Purchases» → молча ничего не происходит → думают, что подписка потеряна. App Review reject.

Исправление: После AppStore.sync(): for await result in Transaction.currentEntitlements { ... handleSuccessfulPurchase(t) }.

MED **M6** PremiumStatusManager.isPremium грузится из UserDefaults каждый launch

□ Services/PremiumStatusManager.swift:112-133

Что не так: loadPersistedState читает rave_user_is_premium из UserDefaults и доверяет. В комбинации с C9 локальный premium state полностью под контролем атакующего.

Влияние: Premium bypass персистит между запусками. Даже при правильном IAP локальный state может расходиться с серверным.

Исправление: Source of truth должен быть серверный User.isPremium (из /api/auth/me). Local cache — только hint.

MED **M7** APIClient.requestNoBody не обрабатывает 404

□ Networking/APIClient.swift:124-131

Что не так: request<T> имеет case 404: throw .notFound, но requestNoBody только 401 и default. Callers не могут отличить «not found» от «server error».

Влияние: leaveRoom на уже удалённой комнате показывает «Ошибка сервера (404): Request failed» вместо «Комната не найдена».

Исправление: Отразить switch из request<T>.

MED **M8** Room.isHost всегда возвращает false (dead computed property)

□ Models/Room.swift:25-28

Что не так: var isHost: Bool { false }. Комментарий врёт: «Set at runtime by ViewModel based on current user» — но никакой код это не делает.

Влияние: Dead/misleading code; потенциальный silent failure, если кто-то начнёт использовать.

Исправление: Удалить property или реализовать: func isHost(userId: String) -> Bool { hostID == userId }.

MED **M9** OrientationManager.isPortrait — неправильная precedence

▢ Utilities/OrientationManager.swift:38-48

Что не так: `first?.interfaceOrientation.isPortrait ?? true && (...)`. `??` имеет ниже precedence, чем `&&`. Парсится как `?? (true && (...))`.

Влияние: OrientationManager.isPortrait возвращает true для некоторых landscape states, когда `UIDevice.current.orientation == .unknown`.

Исправление: Добавить явные скобки: `(first?.interfaceOrientation.isPortrait ?? true) && (...)`.

MED **M10** RoomView вызывает voiceChat.startCall дважды

▢ Views/Room/RoomView.swift:79-90

Что не так: `.task` блок: сначала `await viewModel.joinRoomFlow()` (который сам вызывает `startCall` на строке 97), затем ещё `try? await voiceChat?.startCall(roomId: room.id)`.

Влияние: Wasted work; потенциальная state confusion, если первый вызов упал mid-setup, а второй succeed.

Исправление: Удалить redundant вызов на строке 83.

MED **M11** WebSocketClient.handleReceiveResult реармится на main actor

▢ Networking/WebSocketClient.swift:286-293

Что не так: На `.success receiveMessage()` вызывается снова, что зовёт `socket?.receive { ... Task { @MainActor in handleReceiveResult } }`. Каждый receive hop — один main-actor cycle.

Влияние: Burst из 100 сообщений от сервера сериализуется через 100 main-actor hops, блокируя UI. UI stutter во время chat flood.

Исправление: Process messages на background queue, hop to MainActor только для delegate dispatch.

MED **M12** WebSocketClient.sendRaw error handler — fragile DispatchQueue.main.async

▢ Networking/WebSocketClient.swift:237-246

Что не так: `socket?.send completion` на background queue. `self?.handleDisconnect` — `@MainActor`. `DispatchQueue.main.async` работает, но под strict concurrency может flagged.

Влияние: Strict concurrency warnings; поведение OK, но хрупко.

Исправление: Заменить на `Task { @MainActor [weak self] in ... }`.

MED **M13** FriendManager.init вызывает loadAll() до установки authToken

▢ Services/FriendManager.swift:28-31

Что не так: init() { Task { await loadAll() } }. loadAll → loadFriends → guard api.authToken != nil else { return }. Поскольку FriendManager owns own APIClient (C5), authToken всегда nil.

Влияние: Friends list остаётся пустым на первом launch. Даже после fix C5, если RaveCloneApp конструирует FriendManager ДО завершения restore-токена в AuthService, Task может выполняться раньше.

Исправление: Триггерить loadAll из RaveCloneApp.checkAuth после bridgeAuthToken.

MED **M14** RoomSyncManager.handleAppBackground 30s timeout — stuck .reconnecting

▢ Services/RoomSyncManager.swift:385-397

Что не так: 30s background task hard-disconnects WS. Если OS убивает app до foreground handler, didDisconnectInBackground остаётся true, следующий launch начинается со stale connectionStatus = .reconnecting.

Влияние: Stuck .reconnecting state на cold launch.

Исправление: Reset didDisconnectInBackground = false в init.

MED **M15** HomeView.startCTACollapseTimer Timer в @State без cleanup

▢ Views/Home/HomeView.swift:33, 469-475

Что не так: @State private var ctaCollapseTimer: Timer?. SwiftUI может пересоздать view и reset @State на identity changes. Нет .onDisappear cleanup.

Влияние: Timer fires на dismissed HomeView, возможно выставляя state на stale view.

Исправление: Использовать Task с Task.sleep, либо invalidate в .onDisappear.

MED **M16** WebSocketClient.isConnectedBridge uses MainActor.assumeIsolated

▢ Networking/WebSocketClient.swift:451-455

Что не так: nonisolated var isConnectedBridge: Bool { MainActor.assumeIsolated { self.isConnected } }. assumeIsolated crash-нет, если вызвано из non-MainActor контекста.

Влияние: Crash risk, если non-MainActor код тронет isConnectedBridge.

Исправление: Использовать только с documented precondition, либо предоставить async accessor.

▣ LOW (16) — hygiene / polish / deprecations

LOW L1 Захардкоженные русские строки в views (байпас LocalizationManager)

▣ Views/Home/HomeView.swift (lines 175, 260, 295, 324, 518, 575), RoomCreationView.swift (14 мест), AdminPanelView.swift (8 мест), PrivacySettingsView.swift

Что не так: В приложении есть LocalizationManager с тремя языками (ru/en/zh), но большинство UI обходит его.

Влияние: English и Chinese локализации неполные; переключение языка в settings не обновляет эти экраны.

Исправление: Добавить недостающие ключи в L10n.Key, использовать loc.string(.key).

LOW L2 Room.mockRooms — mock-данные в production

▣ Models/Room.swift:105-143

Что не так: Production-код шлёт 5 фейковых комнат («Дюна 2», «Lo-Fi Chill» и т.д.) как fallback, когда сервер 401 или пустой.

Влияние: Если сервер возвращает empty/401, юзеры видят фейковые «активные» комнаты с фейковым participant count — misleading.

Исправление: Удалить mockRooms или gate behind #if DEBUG.

LOW L3 WSCClient.connectionStats возвращает untyped [String: Any]

▣ Networking/WebSocketClient.swift:428-437

Что не так: Словарь с mixed types (Bool, Int, String). Not Sendable, not type-safe.

Влияние: Любой consumer должен cast; нет compile-time гарантий.

Исправление: Вернуть struct ConnectionStats: Sendable { ... }.

LOW L4 ReactionOverlayView — dead code

▣ Views/Room/ReactionOverlayView.swift

Что не так: Структ существует, есть previews, но никогда не инстанцируется. RoomView использует ReactionSpriteOverlay.

Влияние: ~120 строк мёртвого кода.

Исправление: Удалить файл.

LOW L5 PrivacySettingsView toggles не персистируются и не синхронизируются

▣ Views/Settings/PrivacySettingsView.swift:8-10

Что не так: profileVisibility, onlineStatus, readReceipts — @State без UserDefaults persistence и без backend call.

Влияние: Юзер переключает privacy → уходит с экрана → настройки сбрасываются.

Исправление: Персистить в UserDefaults + POST на /api/users/privacy.

LOW **L6** LoginView Google/Apple sign-in — fake

▢ Views/Auth/LoginView.swift:207-248

Что не так: Тап по Google или Apple показывает spinner, через 1.5s fallback на email form. Реального OAuth flow нет.

Влияние: Misleading UI. Apple Sign In обязателен, если предлагается third-party sign-in (Guideline 4.8).

Исправление: Реализовать ASAuthorizationAppleIDProvider или убрать кнопки.

LOW **L7** AmbilightBackground использует @StateObject для shared singleton

▢ Views/Room/AmbilightBackground.swift:20

Что не так: @StateObject private var sampler = AmbilightSampler.shared. @StateObject означает, что SwiftUI owns lifecycle, но это singleton — два AmbilightBackground будут думать, что они владеют.

Влияние: Conceptual misuse; работает, потому что singleton игнорирует ownership semantics.

Исправление: Использовать @ObservedObject для shared singletons.

LOW **L8** RoomView shareSheet строит неправильный URL

▢ Views/Room/RoomView.swift:119-123

Что не так: URL(string: "https://raveclone.com/join/(room.code)"). Domain — raveclone.com, но DeepLinkRouter.domain — raveclone.app, ShareManager.shareBaseURL — raveclone.app. Path /join/<code>, но DeepLinkRouter распознаёт только /r/<code> или /u/<userId>.

Влияние: Получатели share sheet получают URL, который открывает 404 в браузере.

Исправление: Использовать ShareManager.shareURL(for: room.id, code: room.code).

LOW **L9** EnergyController observer никогда не удаляется

▢ Views/Room/AmbilightBackground.swift:171-181

Что не так: EnergyController.shared регистрирует NotificationCenter observer в init, но никогда не remove-ит. Singleton, поэтому OK на практике.

Влияние: Если класс перестанет быть singleton — leak.

Исправление: deinit { NotificationCenter.default.removeObserver(self) }.

LOW **L10** RaveCloneApp.handleDeepLink хардкодит «Пользователь» для friend invite

▢ RaveCloneApp.swift:135

Что не так: friendInviteAlert = FriendInviteAlert(userId: userId, username: "Пользователь").

Влияние: Username всегда «Пользователь». Реальный username должен быть с сервера.

Исправление: Fetch через FriendManager или /api/users/:id.

LOW **L11** Backend URLs разнесены между Railway и raveclone.app

▢ Services/MediaService.swift:38 vs Networking/APIClient.swift:23

Что не так: APIClient: `https://xpkcakpkfewp-ofewk-pkv-production.up.railway.app/api`.
MediaService: `https://raveclone.app/api`. YouTubeSearchService: `raveclone.app`.
YandexAuthService: `raveclone.app`.

Влияние: Auth/room операции идут на Railway, media extraction — на raveclone.app (которого может не существовать).

Исправление: Centralize base URL в едином Config struct.

LOW **L12** RaveCloneApp.init не пробрасывает auth token в RoomService

▢ RaveCloneApp.swift:43-50

Что не так: RoomService(api: api) разделяет api с AuthService — работает. Но MediaService имеет собственный setAuthToken, обновляется только через bridgeAuthToken(). Если bridgeAuthToken вызывается раньше, чем authService.currentUser populate-ится, getFreshToken() возвращает старый токен.

Влияние: Media operations могут работать с истёкшим токеном.

Исправление: Subscribe MediaService.setAuthToken на token-change publisher в AuthService.

LOW **L13** AuthService.deleteAccount не удаляет на самом деле

▢ Services/AuthService.swift:111-114

Что не так: `func deleteAccount() async throws { /* TODO: добавить DELETE /api/auth/me */ try await signOut() }`.

Влияние: GDPR/CCPA right-to-delete violation. Данные аккаунта персистируют на сервере.

Исправление: Реализовать DELETE /api/auth/me на backend, вызывать перед signOut.

LOW **L14** MarqueeMessageView.width использует NSString.size

▢ Views/Room/MarqueeMessageView.swift:64-67

Что не так: NSString.size(withAttributes:) возвращает approx size, не учитывает line breaks или emoji rendering differences. Используется для вычисления scrollDistance.

Влияние: Minor visual glitch на длинных сообщениях с emoji.

Исправление: Использовать Text(...).measureSize() via PreferenceKey.

LOW

L15

WebSocketClient.connectionStats — force-unwrap after nil-check

□ Networking/WebSocketClient.swift:435

Что не так: notifyConnectedIfNeeded (строка 421) делает Logger.ws.info("Restoring room session: \(activeRoomID!)") после if activeRoomID != nil check. Force-unwrap безопасен, но fragile при рефакторинге.

Влияние: Crash risk при рефакторинге без сохранения nil-check.

Исправление: Использовать if let binding.

LOW

L16

SyncEngine.deinit трогает @MainActor state

□ Services/SyncEngine.swift:88-92

Что не так: deinit { player?.pause(); if let observer = timeObserver { player?.removeTimeObserver(observer) } }. player и timeObserver — @MainActor ivars. deinit — nonisolated по умолчанию в Swift 5.10+.

Влияние: Concurrency violation под strict concurrency.

Исправление: Mark deinit nonisolated явно + MainActor.assumeIsolated, либо move teardown в cleanup() method.

04 Backend (Node.js / Fastify) — 34 бага

Аудит 24 файлов: index, config, middleware/auth, websocket manager+handler, 7 routes (auth, auth-social, rooms, messages, friends, admin, media-v2, profile), services (push, youtube, extractors), Prisma schema, Dockerfile, env.example.

CRITICAL (5) — security holes

CRIT

B1

Admin-роуты никогда не аутентифицируются — весь admin panel возвращает 401

server/src/routes/admin.ts:24-47

Что не так: Регистрируется preHandler, который читает request.user, но никогда не вызывает fastify.authenticate. @fastify/jwt не авто-verify — request.user всегда undefined, каждый admin-запрос попадает в if (!user) return reply.status(401).

Влияние: Весь admin panel неработоспособен. prisma.user.findUnique({ where: { id: user.id } }) на строке 34 — dead code. Бан/модерация не работают.

Исправление: Добавить { preHandler: [fastify.authenticate] } на каждый admin route, либо вызывать await request.jwtVerify() внутри hook.

CRIT

B2

request.user.sub === undefined в messages/friends/profile — 12 сломанных хендлеров

server/src/middleware/auth.ts:54 + messages.ts:7,15 + friends.ts:15,27,33,45,52 + profile.ts:7,13,21,45,52

Что не так: Middleware ставит request.user = { id, username, email } (без sub). Но 12 хендлеров читают request.user.sub. Для create-вызовов (DM, FriendRequest, Friendship, WatchHistory) Prisma бросает — поле non-null → 500. Для findMany/count вызовов where: { userID: undefined } молча матчит все строки.

Влияние: Каждый DM send, friend request, friend accept, watch-history save, premium fetch, subscription create падает с 500. Для list-запросов — данные других юзеров видны всем.

Исправление: Заменить каждое request.user.sub на request.user.id.

CRIT

B3

Чат-имперсонация — senderID/senderName берутся из payload клиента

server/src/websocket/ws-manager.ts:634-651

Что не так: ChatMessage.create({ senderID: msg.senderID, ... }) — senderID берётся из клиентского payload. Broadcast: { ...msg, text, id, timestamp } спредит senderID, senderName, senderRole из клиента.

Влияние: Любой юзер может персистить чаты под чужим ID и бродкастить их как «Admin»/«system» всей комнате.

Исправление: Игнорировать клиентские identity fields; использовать conn.user.id, conn.user.username, conn.user.role.

CRIT**B4****Banned-юзеры могут signin и подключаться по WS**

□ server/src/routes/auth.ts:77-120 + auth-social.ts:50-130 + ws-handler.ts:111-143

Что не так: signin и auth-social никогда не читают bannedUntil перед выдачей JWT. authenticateWs verify-ит JWT signature, но не ищет юзера в DB — даже удалённые юзеры продолжают работать. Admin bans ставят bannedUntil и зовут disconnectUserEverywhere, но JWT остаётся валидным 7 дней, юзер может сразу re-signin.

Влияние: Баны не работают. Забаненный юзер заходит через 5 секунд.

Исправление: В authenticate и authenticateWs fetch юзера из DB, reject если bannedUntil > now. Добавить server-side jti blacklist или short-lived access + refresh tokens.

CRIT**B5****Free premium bypass — POST /api/users/me/create-subscription**

□ server/src/routes/profile.ts:51-58

Что не так: Любой авторизованный юзер POST { plan: "monthly" } → сервер ставит isPremium: true, premiumUntil: now + 30d. Нет Stripe/RevenueCat/App-Store-Server webhook, нет signature check, нет receipt validation.

Влияние: Полный обход платной подписки. Любой может получить premium бесплатно.

Исправление: Удалить self-service upgrade route; isPremium ставить только из verified payment-provider webhook.

■ HIGH (11) — race / leaks / SSRF / arg-injection

HIGH B6 Несколько new PrismaClient() — bypass singleton

□ messages.ts:3 + friends.ts:3 + profile.ts:3

Что не так: const prisma = new PrismaClient() в трёх файлах, минуя config/db.ts singleton.

Влияние: Лишние connection pools, Postgres exhaustion под нагрузкой.

Исправление: Импортировать shared prisma из config/db.ts.

HIGH B7 JWT в URL query string для WS

□ ws-handler.ts:115-116

Что не так: new URL(req.url).searchParams.get('token'). JWT в URL утечёт в proxy logs, browser history, Referer header.

Влияние: Credential disclosure через логи.

Исправление: Использовать Sec-WebSocket-Protocol subprotocol или first-message auth.

HIGH B8 authenticateWs хардкодит role: "USER"

□ ws-handler.ts:124,134

Что не так: const user = { id: payload.sub, username: payload.username, role: 'USER' }. Все admin WS-фичи (ws-manager.ts:197-220 admin-join broadcast) — dead code.

Влияние: Админ-функционал через WS недоступен. Невозможно модерировать через WS.

Исправление: Fetch юзера из DB, populate реальный role.

HIGH B9 disconnectAll() вызывается, но не определена

□ index.ts:124-128 vs ws-manager.ts

Что не так: В index.ts SIGTERM handler вызывает wsManager.disconnectAll(), но grep подтверждает — такого метода в ws-manager.ts нет.

Влияние: На SIGTERM WS-клиенты не получают close frame, heartbeat interval продолжает стрелять, process падает.

Исправление: Добавить реальный disconnectAll() method.

HIGH B10 SSRF в /api/media/extract и /probe

□ services/extractors/web.ts:65,71,83 + mediaExtractor.ts:160-162

Что не так: fetch(url) на attacker-supplied URLs без allowlist. Catch-all web fallback для любого URL.

Влияние: Атака на AWS IMDSv1, внутренние сервисы, localhost DB probes.

Исправление: Private-IP blocklist после DNS lookup, redirect cap, body size limit, удалить catch-all web fallback.

HIGH **B11** detectSource использует lower.includes(d) для domain matching

□ mediaExtractor.ts:149-151

Что не так: lower.includes(d) — bypassable через https://evil.com/?x=netflix.com.

Влияние: Неверная классификация источника.

Исправление: Парсить через new URL(), сравнивать hostname.

HIGH **B12** Argument injection в yt-dlp — URL без --

□ youtube.ts:100-110, 261-270 + vk.ts:24-29 + rutube.ts:25-30

Что не так: URL передаётся как trailing arg без -- сепаратора. Атакующий может передать --exes=... или --plugin-dirs=... как URL.

Влияние: RCE-adjacent. Выполнение произвольных команд через yt-dlp plugins.

Исправление: Вставить "--" перед URL arg.

HIGH **B13** /ws/stats debug endpoint без аутентификации

□ ws-handler.ts:38-40

Что не так: Эндпоинт отдаёт total connections, room IDs, host IDs, participant counts без auth.

Влияние: Утечка чувствительной инфо — кто в каких комнатах, сколько активных юзеров.

Исправление: Gate behind admin auth или удалить в prod.

HIGH **B14** JWT_SECRET defaults to "dev-secret-change-me", CORS_ORIGIN = "*" + credentials: true

□ config/index.ts:37,43 + index.ts:48

Что не так: JWT_SECRET = process.env.JWT_SECRET ?? 'dev-secret-change-me'. CORS_ORIGIN = ['*'] с credentials: true.

Влияние: Пропущенные env vars в prod → куётся любой JWT + reflected-origin CORS.

Исправление: required('JWT_SECRET'); reject ['*'] когда credentials: true.

HIGH **B15** Guest username collision после 9000 аккаунтов

□ auth-social.ts:113

Что не так: Raver_\${1000..9999} с @unique. После 9000 гостей → P2002 → 500. Нет per-IP cap на /api/auth/guest.

Влияние: DB pollution; сервис падает после 9000 гостей.

Исправление: Использовать nanoid suffix; stricter per-IP rate limit на /guest.

HIGH

B16

VK token verification не доказывает владение приложением

□ auth-social.ts:300-307

Что не так: users.get принимает любой валидный VK token из любого app. Email никогда не возвращается users.get (требуется OAuth scope), поэтому upsertSocialUser всегда синтезирует email → несколько строк на VK-юзера.

Влияние: Любой VK token логинит. Дубликаты пользователей.

Исправление: Использовать VK secure.checkToken с VK_CLIENT_SECRET; email только из OAuth callback.

🔴 MEDIUM (10) + 🟡 LOW (6)

🔴 r B17 equest.user.role === undefined в regular routes

📄 auth.ts:47

Что не так: select: { id: true, username: true, email: true } — без role. request.user.role всегда undefined в regular route handlers.

Влияние: Role-based checks в regular routes не работают.

Исправление: Добавить role: true в select.

🟡 a B18 authenticate трактует только "Unauthorized" как 401, всё остальное 500

📄 auth.ts:55-61

Что не так: catch (err) { if (err.message === 'Unauthorized') return 401; return 500 }. Expired tokens → 500.

Влияние: Юзер видит «Ошибка сервера» вместо «Сессия истекла, войдите снова».

Исправление: Catch FAST_JWT_* error codes, возвращать 401.

🟡 M B19 AX_ROOM_PARTICIPANTS = 20 глобальный cap игнорирует per-room maxParticipants

📄 ws-manager.ts:191

Что не так: if (state.participants.size >= MAX_ROOM_PARTICIPANTS) — глобальный cap 20. room.maxParticipants: 2 всё равно допускает 20 WS-коннектов.

Влияние: Premium-комнаты с caps 50 работают как 20-сар; приватные 2-человечные комнаты могут принять 20.

Исправление: Load room.maxParticipants в RoomRuntimeState.

🟡 H B20 ет host migration — host disconnect сразу деактивирует комнату

📄 ws-manager.ts:245-260

Что не так: При дисконнекте хоста комната сразу деактивируется. Network blip kills room для всех гостей.

Влияние: Комната «умирает» при кратковременной потере сети у хоста.

Исправление: Promote longest-tenured member или 60s grace period.

🟡 c B21 hatRateLimits map никогда не чистится

📄 ws-manager.ts:73

Что не так: Map<userId, { count, resetAt }> — одна запись на уникального юзера навсегда.

Влияние: Memory leak — карта растёт бесконечно.

Исправление: Delete в unregister когда последний conn закрылся; использовать lru-cache.

□ c **B22** `leanupRoom` вызывает `redis.del` без `await/.catch()``ws-manager.ts:288-293`

Что не так: `redis.del(...); redis.del(...)` — fire-and-forget. Если Redis упал → unhandled rejection → process crash (Node 15+ default).

Влияние: Process crash при потере Redis.

Исправление: `await Promise.all([...]).catch(...)`.

□ R **B23** `edis SREM` на `disconnect` удаляет юзера из `participant set` даже если у него есть другой открытый коннект`ws-manager.ts:243`

Что не так: SET дедуплицирует по value, поэтому второй коннект невидим для cross-instance reads.

Влияние: Юзер с двумя вкладками: при закрытии одной вторая становится «невидимой» для других инстансов.

Исправление: Только SREM когда нет remaining connID для этого юзера в roomID.

□ c **B24** `onnID = ws_${user.id}_${Date.now()}` — коллизия при коннекте в same ms`ws-manager.ts:88`

Что не так: `const connID = `ws_${user.id}_${Date.now()}``. Если тот же юзер коннектится дважды в одну ms, первый conn orphan-ится с live handlers но unregistered.

Влияние: Lost connection, leak handlers.

Исправление: Использовать `nanoid()`.

□ H **B25** `heartbeart interval == timeout` (оба 45s)`config/index.ts:50 + ws-manager.ts:75`

Что не так: `checkHeartbeats` запускается каждые 45s, `timeout` тоже 45s. Dead conn detected в 45-90s.

Влияние: Долгое обнаружение мёртвых соединений.

Исправление: Запустить `checkHeartbeats` каждые 30s.

□ H **B26** нет global error handler — Zod errors leak stack traces`index.ts`

Что не так: Нет `setErrorHandler` нигде (grep confirmed).

Влияние: Zod errors возвращают stack trace клиенту — info disclosure.

Исправление: Добавить Zod-aware error handler, возвращающий consistent { error, issues }.

H **B27** `ет unhandledRejection/uncaughtException handlers в index.ts`

`index.ts`

Что не так: С многими fire-and-forget promises одна rejection убивает process.

Влияние: Process crash на unhandled rejection.

Исправление: Register оба, log + graceful shutdown.

I **B28** `oadConfig(console as any) — _log unused`

`config/index.ts:31`

Что не так: Параметр передаётся, но не используется.

Влияние: Dead parameter.

Исправление: Удалить параметр.

g **B29** `enerateRoomCode modulo bias`

`utils/index.ts:12`

Что не так: `chars[Math.floor(Math.random() * chars.length)]` — modulo bias.

Влияние: Неравномерное распределение кодов комнат.

Исправление: `crypto.randomInt(0, chars.length)`.

f **B30** `astify.jwt.sign(payload, { sub: user.id })` — второй аргумент wrong type

`auth.ts:58-60 + auth-social.ts:173-176`

Что не так: Передаётся second arg неверного типа, молча игнорируется.

Влияние: sub не устанавливается в JWT payload.

Исправление: Удалить второй arg, добавить sub в payload объект.

D **B31** `ead code: youtubeService decorated в index.ts:69-73, но media-v2.ts:26 создаёт свой`

`index.ts + media-v2.ts`

Что не так: Декорация бесполезна — каждый route создаёт свой инстанс.

Влияние: Лишний код + несогласованная конфигурация.

Исправление: Удалить декорацию или использовать её.

S

B32

chema integrity gaps: PlaybackState.roomID и WatchHistory.roomID без FK к Room

schema.prisma:240,222

Что не так: Нет foreign key к Room. Позволяет баги B17 (произвольный roomID в POST /me/history).

Влияние: roomID в playback state и watch history.

Исправление: Добавить room Room? @relation(...) к обеим моделям.

U

B33

serRole enum имеет MODERATOR и FOUNDER — ни один code path не читает

schema.prisma:14-19

Что не так: Значения объявлены, но не используются.

Влияние: Dead schema.

Исправление: Реализовать room-scoped permissions или удалить.

P

B34

OST /api/rooms/:id/playback без host-check и без Zod validation

rooms.ts:484-502

Что не так: body = request.body as { time, isPlaying } — raw cast. Нет findUnique, нет проверки room.hostID === user.id, нет zod-схемы. PlaybackState не имеет FK к Room.

Влияние: Любой авторизованный юзер может спамить upsert для любого roomID (включая случайные строки).

Исправление: Zod-validate body, findUnique room, требовать room.hostID === user.id, добавить FK в schema.

05 React Native (Expo) — 61 баг

Аудит 31 файла: App.tsx, navigation, auth store, WS/WebRTC/DRM/ScreenShare services, 7 screens, 5 components, native manifests/plists.

❑ CRITICAL (8) — security / auth bypass

CRIT R1 Ngrok auth token захардкожен в VCS

❑ mobile/scripts/patch-ngrok.js:18

Что не так: '3Fk4HLUubHn0SxiisuL6tbuiSom_41ehdA3NACxFaNNEZ8F3m' прямо в коде.

Влияние: Утечка credentials. Любой может использовать ваш ngrok-аккаунт.

Исправление: Читать из process.env.NGROK_AUTH_TOKEN; немедленно ротировать утёкший token.

CRIT R2 JWT в plain AsyncStorage

❑ mobile/src/store/authStore.ts:43-44, 55, 78, 87

Что не так: AsyncStorage.setItem('auth_token', token). AsyncStorage — нешифрованное SQLite/plist.

Влияние: Token disclosure → account takeover.

Исправление: Использовать expo-secure-store (SecureStore.setItemAsync с WHEN_UNLOCKED).

CRIT R3 DRM-куки Netflix/Кинопоиск в AsyncStorage

❑ mobile/src/services/DrmSessionManager.ts:91, 179

Что не так: Session cookies платных сервисов в нешифрованном хранилище.

Влияние: Утечка сессий платных сервисов — кто угодно с доступом к filesystem может использовать ваш Netflix.

Исправление: Перенести куки в SecureStore keyed per serviceID; в AsyncStorage держать только metadata.

CRIT R4 Offline-guest fallback — обход auth gate

❑ mobile/src/screens/AppAuthScreen.tsx:108-118

Что не так: При network blip: stores synthetic offline_* token, flips isAuthenticated: true.

Влияние: Полный обход auth gate на любой сетевой проблеме. Любой может «войти» офлайн.

Исправление: Удалить fallback; показывать «Network unavailable» error.

CRIT **R5** Demo exchange — trivial auth bypass

▢ mobile/src/screens/AppAuthScreen.tsx:144, 212

Что не так: Alert OK button: exchange("google", "google", { idToken: "demo_google_token" }). То же для VK.

Влияние: Тривиальный auth bypass — любой может залогиниться как google-юзер с демо-токеном.

Исправление: Удалить demo exchange; показывать configuration error.

CRIT **R6** console.log URL с JWT — утечка в logcat

▢ mobile/src/services/wsService.ts:165

Что не так: console.log('[WS] Connecting to', url) где url содержит ?token=<JWT>.

Влияние: JWT утекает в logcat (Android) / Xcode console (iOS). Любой с доступом к logs может угнать аккаунт.

Исправление: Логировать только host: console.log('[WS] Connecting to', WS_URL).

CRIT **R7** Deep-link scheme raveclone://* без host/path — phishing

▢ mobile/android/app/src/main/AndroidManifest.xml:25-30 + ios/RaveClone/Info.plist:25-34

Что не так: Scheme raveclone:// зарегистрирован без host/path. Любая raveclone://* ссылка открывает приложение.

Влияние: Phishing-вектор — злоумышленник может прислать ссылку, которая откроет приложение в произвольном состоянии.

Исправление: Ограничить: <data android:scheme="raveclone" android:host="room" android:pathPrefix="/join"/>.

CRIT **R8** GOOGLE_CLIENT_ID = placeholder
YOUR_GOOGLE_CLIENT_ID.apps.googleusercontent.com

▢ mobile/src/screens/AppAuthScreen.tsx:35, 38

Что не так: И client ID, и redirect URI используют literal placeholder. Google OAuth не может работать в принципе.

Влияние: Google sign-in полностью неработоспособен.

Исправление: Читать из Constants.expoConfig.extra.googleClientId.

■ HIGH (21) — leaks / non-functional features

HIGH **R9** `navigateBack` читает несуществующий `route.params.navigation`

□ `RoomScreen.tsx:193-197`

Что не так: `route.params.navigation` не существует. Alert OK no-op.

Влияние: Кикнутые/закрытые пользователи застревают на комнате.

Исправление: Использовать `useNavigation` hook.

HIGH **R10** `RoomPlayer` кормит YouTube watch URL в expo-av Video

□ `RoomPlayer.tsx:138`

Что не так: `NativePlayer` feeds `https://www.youtube.com/watch?v=...` в expo-av Video.

Влияние: YouTube не играет — expo-av не умеет YouTube watch URLs.

Исправление: Использовать `WebView` для YouTube или YouTube IFrame API.

HIGH **R11** `WebViewPlayer` грузит `media.streamURL` вместо `webViewBaseURL`

□ `RoomPlayer.tsx:218`

Что не так: Загружается прямой URL видеофайла вместо `webViewBaseURL`.

Влияние: DRM-сервисы 404 / блокируют запрос.

Исправление: Грузить `webViewBaseURL`, не `media.streamURL`.

HIGH **R12** `injectJavaScript` только на page load — guest не получает seek-команды

□ `RoomPlayer.tsx:219, 233-234`

Что не так: `injectJavaScript` выполняется только при load. Последующие seek-команды не доходят до guest.

Влияние: Гость постоянно десинхронизирован после первой перемотки.

Исправление: Использовать `webViewRef.injectJavaScript(code)` для runtime-команд.

HIGH **R13** Host шлёт WebSocket seek flood каждые 500ms

□ `RoomPlayer.tsx:118-123`

Что не так: `onPlaybackStatusUpdate` вызывает `onPositionChange` каждые ~500 ms → WebSocket seek flood.

Влияние: Сервер заваливается seek-командами; гости дёргаются.

Исправление: Throttle на 2-3s, либо слать только при реальном seek.

HIGH R14 pc.connectionState === 'disconnected' триггерит handlePeerLeft

□ VoiceChatService.ts:226-231

Что не так: Transient network blip permanently mute peers.**Влияние:** Кратковременная потеря сети навсегда мьютит пира — голос не восстанавливается.**Исправление:** Act only on "failed", не на "disconnected".**HIGH R15** Mute button — TODO no-op

□ VoiceChatPanel.tsx:64-70

Что не так: setIsMuted flips UI, но никогда не вызывает voiceChat.toggleMute().**Влияние:** Микрофон остаётся горячим. Юзер думает, что замьютился — на самом деле нет.**Исправление:** Вызывать voiceChat.toggleMute() в обработчике.**HIGH R16** webrtc_leave шлёт roomId: "" — server не может разорвать p2p

□ VoiceChatService.ts:175-179

Что не так: emit('webrtc_leave', { roomId: "" }) — пустой roomId.**Влияние:** Remote peer's pc никогда не teardown-ится при выходе из комнаты.**Исправление:** Передавать реальный roomId.**HIGH R17** ScreenShareService — мёртвый код

□ ScreenShareService.ts (весь файл)

Что не так: Нет wsService.on(...) listeners; handleSignaling unreachable. RoomScreen button просто alert-ит.**Влияние:** Фича демонстрации экрана полностью неработоспособна.**Исправление:** Wire up signaling listeners, либо удалить фичу.**HIGH R18** ICE config без TURN server

□ ScreenShareService.ts:24-33

Что не так: TURN закомментирован как placeholder.**Влияние:** Screen share fails behind symmetric NAT.**Исправление:** Добавить реальный TURN server (Twilio/coturn).

HIGH **R19** WS effect deps [roomId] only — stale closures

□ RoomScreen.tsx:60-190

Что не так: `useEffect(..., [roomId])` — `voiceChat`, `user`, `navigateBack` в closure устаревают.**Влияние:** Stale state, неверные handlers после re-render.**Исправление:** Добавить все используемые переменные в deps, либо refactor.**HIGH** **R20** Sync effect missing `isHost` и `applyPosition` deps

□ RoomPlayer.tsx:73-79

Что не так: Effect не перезапускается при смене хоста.**Влияние:** Host-transfer ломает sync.**Исправление:** Добавить `isHost`, `applyPosition` в deps.**HIGH** **R21** `applyPosition` async после unmount — native crash

□ RoomPlayer.tsx:81-92

Что не так: `video.playAsync()` вызывается после unmount.**Влияние:** Native crash risk на React Native 0.74+.**Исправление:** Использовать `isMountedRef` или `AbortController`.**HIGH** **R22** WS onclose всегда реконнектит — kicked users в цикле

□ wsService.ts:191-200

Что не так: onclose всегда schedules reconnect, без проверки code.**Влияние:** Кикнутые пользователи переподключаются и re-joinят в цикле — DDoS собственного сервера.**Исправление:** Проверять close code; не реконнектить при 4001 (kicked) / 4003 (banned).**HIGH** **R23** `send()` молча дропает сообщения если WS не OPEN

□ wsService.ts:264-269

Что не так: Нет queue, нет retry. Если WS connecting/closing, сообщение теряется.**Влияние:** Join/chat/sync команды потеряны при reconnect.**Исправление:** Implement message queue с flush на connected event.

HIGH **R24** setTimeout 500ms race для joinRoom

□ RoomScreen.tsx:179-181

Что не так: setTimeout(() => wsService.joinRoom(roomID), 500). Если WS занимает >500ms, join дропается (см. R23).

Влияние: Юзер не joinит комнату при медленном коннекте.

Исправление: Subscribe на connected event вместо таймера.

HIGH **R25** JSON.parse(event.nativeEvent.data) без try/catch

□ DrmOverlay.tsx:171-176

Что не так: Любой non-JSON postMessage из загруженной страницы крашит React tree.

Влияние: Crash от любого malformed message из WebView.

Исправление: Wrap в try/catch, логировать raw data.

HIGH **R26** Нет global ErrorBoundary

□ App.tsx

Что не так: Любой render exception → белый экран без recovery.

Влияние: Юзеры видят белый экран при любом рендер-баге, без возможности сообщить.

Исправление: Добавить ErrorBoundary с fallback UI.

HIGH **R27** ContentBrowserScreen fetch-ит результаты, но не отображает

□ ContentBrowserScreen.tsx:61-78

Что не так: Search fetch-ит results и console.log count, но FlatList рендерит mocks.

Влияние: Поиск не работает — юзер видит static mocks вместо реальных результатов.

Исправление: Сохранять results в state, рендерить в FlatList.

HIGH **R28** CreateRoomModal — showCreate никогда не true

□ HomeScreen.tsx:70, 242

Что не так: CreateRoomModal отрендерен, но showCreate никогда не выставляется в true.

Влияние: Dead code, unreachable UI element.

Исправление: Либо удалить, либо добавить trigger.

HIGH**R29**

DEV_URL = localhost:3000 — недостижим с эмулятора/устройства

□ src/config/index.ts:20

Что не так: http://localhost:3000 — Android emulator требует 10.0.2.2, физические устройства — IP хоста.

Влияние: Backend unreachable в dev на Android.

Исправление: Использовать 10.0.2.2 для Android emulator, IP хоста для устройств.

🔴 MEDIUM (17) + 🟡 LOW (15)

🔴 O R30 опасные permissions в AndroidManifest

📄 AndroidManifest.xml:4,6,8

Что не так: READ_EXTERNAL_STORAGE, SYSTEM_ALERT_WINDOW, WRITE_EXTERNAL_STORAGE — неиспользуемые и опасные.

Влияние: Google Play отклонит за лишние permissions.

Исправление: Удалить unused permissions.

🔴 D R31 deprecated BLUOTH permission, missing BLUETOOTH_CONNECT/SCAN

📄 app.json:33

Что не так: BLUETOOTH deprecated. Нет BLUETOOTH_CONNECT/BLUETOOTH_SCAN для Android 12+.

Влияние: Bluetooth-функции не работают на Android 12+.

Исправление: Заменить на BLUETOOTH_CONNECT, BLUETOOTH_SCAN.

🔴 S R32 hare message «Plink!» вместо «RaveClone»

📄 RoomScreen.tsx:203

Что не так: Copy-paste leftover.

Влияние: Несогласованный брендинг.

Исправление: Унифицировать имя.

🔴 s R33 howScreenShareBtn state, никогда не читается/пишется

📄 RoomScreen.tsx:46

Что не так: Dead state.

Влияние: Wasted memory, confusing code.

Исправление: Удалить или использовать.

🔴 E R34 mpty catch {} everywhere

📄 HomeScreen.tsx:78, MyRoomsScreen.tsx:32, ContentBrowserScreen.tsx:73,106,125, ProfileScreen.tsx:57, RoomScreen.tsx:205

Что не так: catch {} без обработки.

Влияние: Юзер не видит ошибок; debugging невозможно.

Исправление: Логировать + показывать toast.

Р **R35** public OpenRelay TURN credentials захардкожены

□ VoiceChatService.ts:50-52

Что не так: Публичные TURN creds — abuse-prone, unreliable, не для prod.

Влияние: TURN server абыюзят, unreliable в prod.

Исправление: Self-hosted coturn или Twilio TURN.

и **R36** seAuthStore() подписывается на весь store

□ AppNavigator.tsx:48, HomeScreen.tsx:66, ProfileScreen.tsx:20

Что не так: useAuthStore() без selector — re-рендер на любое изменение store.

Влияние: Unnecessary re-renders, perf degradation.

Исправление: Использовать selectors: useAuthStore(s => s.user).

Ј **R37** SON.parse(userValue) as AppUser — без runtime validation

□ authStore.ts:60

Что не так: JSON.parse(userValue) as AppUser — cast без проверки.

Влияние: Corrupted storage crashes app on hydrate.

Исправление: Использовать zod schema для validation.

Ј **R38** SON.parse(value) as DrmSession — то же

□ DrmSessionManager.ts:142

Что не так: Corrupted cookie blob crashes hydrate.

Влияние: Crash on bad storage.

Исправление: Zod validation.

Е **R39** mail validation !email.includes("@") принимает "a@b"

□ AppAuthScreen.tsx:225

Что не так: Слабая валидация. Password min 6 ниже NIST recommendation (8).

Влияние: Слабые пароли, фейковые emails.

Исправление: Строгая regex + min 8 chars.

F **R40** latList renderItem не memoized, ChatBubble не React.memo

□ ChatView.tsx:122-129

Что не так: Re-render всех сообщений на каждое новое.

Влияние: Perf degradation в длинных чатах.

Исправление: React.memo для ChatBubble, useCallback для renderItem.

Р R41 an gesture без activeOffsetX/failOffsetY — hijacks vertical scroll

ChatView.tsx:68-87

Что не так: Horizontal pan hijacks vertical scroll message list.**Влияние:** Скролл сообщений блокируется свайпом.**Исправление:** Добавить activeOffsetX, failOffsetY.**К R42** eboardAvoidingView behavior={undefined} на Android

AppAuthScreen.tsx:252

Что не так: Behavior undefined на Android — клавиатура перекрывает inputs.**Влияние:** Юзер не видит email/password под клавиатурой.**Исправление:** behavior={Platform.OS === 'ios' ? 'padding' : 'height'}.**о R43** nError={e => setError(`...\${e}`)} рендерит [object Object]

RoomPlayer.tsx:145

Что не так: String template с object — [object Object].**Влияние:** Бессмысленное error message.**Исправление:** JSON.stringify(e) или extract message.**О R44** perator precedence: '...' + true ? 'HOST' : 'GUEST'

RoomPlayer.tsx:320

Что не так: Всегда вычисляется в 'HOST'.**Влияние:** Неверная бизнес-логика.**Исправление:** Добавить скобки: ('...' + true) ? 'HOST' : 'GUEST'.**и R45** seEffect для googleResponse missing exchange в deps

AppAuthScreen.tsx:80-84

Что не так: Stale closure для exchange function.**Влияние:** Старая версия функции вызывается.**Исправление:** Добавить exchange в deps.**н R46** ew MediaStream() — polyfill missing в Expo Go

VoiceChatService.ts:191, 273

Что не так: new MediaStream() полагается на global, который Expo Go mock не polyfill-ит.**Влияние:** Latent ReferenceError в Expo Go.**Исправление:** Импортировать из react-native-webrtc.

Н R47 ardcoded prod URL xpkcakpkfewp-ofewk-pkv-production.up.railway.app

config/index.ts:17

Что не так: URL Railway в коде.

Влияние: Если Railway domain изменится — сломается.

Исправление: Читать из env.

а R48 ppleId/appleTeamId placeholders в eas.json

eas.json:32,34

Что не так: Placeholders в build config.

Влияние: Build упадёт на CI.

Исправление: Подставить реальные значения.

G R49 oogle/VK client ID placeholders

AppAuthScreen.tsx:35,201

Что не так: То же, что R8.

Влияние: OAuth не работает.

Исправление: Подставить реальные client IDs.

о R50 wner: "rageultimate" в app.json

app.json:49

Что не так: Лишний handle в Expo config.

Влияние: Expo project ownership раскрыт.

Исправление: Убрать или заменить.

е R51 хро-av deprecated в SDK 54+

package.json:21

Что не так: expo-av deprecated → migrate to expo-video.

Влияние: Будущие SDK не поддержат.

Исправление: Мигрировать на expo-video.

о R52 fferToReceiveAudio/Video deprecated

VoiceChatService.ts:234, ScreenShareService.ts:253

Что не так: Deprecated WebRTC API.

Влияние: Будущие версии react-native-webrtc не поддержат.

Исправление: Использовать addTransceiver.

i **R53** `import { ScrollView } в конце файла``ChatView.tsx:202`**Что не так:** Импорт после первого использования.**Влияние:** Confusing code style.**Исправление:** Переместить наверх.**I** **R54** `nconsistent React.useEffect vs imported useEffect``HomeScreen.tsx:85, MyRoomsScreen.tsx:40`**Что не так:** Mixed style.**Влияние:** Inconsistent code style.**Исправление:** Унифицировать.**c** **R55** `onsole.log/console.warn в production paths``Multiple`**Что не так:** Логи в prod путях.**Влияние:** Performance + info leak.**Исправление:** Удалить или заменить на `__DEV__` check.**a** **R56** `s any casts на navigation``HomeScreen.tsx:170, ServicePickerScreen.tsx:46, ContentBrowserScreen.tsx:123,137, RoomScreen.tsx:194`**Что не так:** as any casts.**Влияние:** Type safety lost.**Исправление:** Правильно типизировать navigation.**t** **R57** `ext.length * fontSize * 0.65 — wrong for Cyrillic/emoji``AnimatedGradientText.tsx:56`**Что не так:** Width estimate неверный для кириллицы и emoji.**Влияние:** Marquee glitch на non-Latin текстах.**Исправление:** Использовать text measurement.**U** **R58** `nused loop var userID``VoiceChatService.ts:112`**Что не так:** Loop variable не используется.**Влияние:** Dead code.**Исправление:** Удалить.

t **R59** sconfig без strict, без path aliases

▢ tsconfig.json

Что не так: strict не включён, нет path aliases.

Влияние: Type safety ниже возможного, относительные импорты.

Исправление: Включить strict, добавить paths.

H **R60** ardcoded v1.0.0 в ProfileScreen, app.json говорит 1.1.0

▢ ProfileScreen.tsx:106

Что не так: Несогласованность версий.

Влияние: Юзер видит неправильную версию.

Исправление: Читать из app.json.

o **R61** ff() method определён, но не используется

▢ wsService.ts:286-288

Что не так: Dead code.

Влияние: Wasted lines.

Исправление: Удалить или использовать.

06

Редизайн: премиум дизайн-система «Cinema Violet»

Текущая палитра приложения (Color+Theme.swift) построена на бирюзовом акценте (#00C2FF) и неоновом изумрудном (#00FF88). Для premium watch-party проекта это визуально дешёво и не соответствует позиционированию. Конкуренты (Rave, Hearo, Disney+, Apple Music) используют тёплые глубокие тона — фиолетовый, магента, золото. Ниже — полная замена дизайн-системы с разделением ролей (free / premium / host / admin) и готовым кодом для drop-in миграции.

6.1 Почему бирюзовый — это дешёво для premium-продукта

Бирюзовый (#00C2FF) и неоново-зелёный (#00FF88) — это палитра бюджетных streaming-сервисов и IoT-приложений. В контексте «смотрим вместе» пользователь ожидает ощущение кинозала или лонж-бара: глубокие тёмные тона, точечные тёплые подсветки, премиум-акценты. Бирюзовый фон подсознательно ассоциируется с дешёвой Android утилитой, а не с премиум-подпиской за 299₽/мес.

Дополнительно: текущая система не различает роли визуально. У premium-юзера и у free-юзера одинаковые аватарки, одинаковые кнопки, одинаковые карточки. Нет ощущения «апгрейда» — а именно это и продаёт подписку.

6.2 Конкурентный анализ

Продукт	Палитра	Premium-сигнал	Урок
Rave (оригинал)	Глубокий фиолетовый + неон	Премиум-цвета в фоне	Фиолетовый = бренд
Hearo	Тёмный серый + точечные акценты	Минимализм	Меньше = дороже
Disney+	Тёмно-синий + золотые акценты	Золото на СТА	Золото = premium
Apple Music	Розово-красный gradient на чёрном	Animated gradient	Движение = жизнь
Spotify	Зелёный на чёрном (иконичный)	—	Не для watch-party
Telegram	Голубой + синий	Premium: звёздное сияние	Premium ring

Вывод: Cinema Violet = глубокий фиолетовый (как Rave) + золото для premium (как Disney+) + магента как эмоциональный акцент (как Apple Music). Это даёт ощущение премиум-кинотеатра.

6.3 Новая палитра «Cinema Violet»

Роль	Имя	Hex	Замена
------	-----	-----	--------

Акцент	ravePrimary	● #A855F7	Бирюзовый #00C2FF → фиолетовый
Акцент	raveAccent	● #EC4899	Розовый #FF2D87 → магента (более насыщенная)
Premium	ravePremium	● #FBBF24	НОВЫЙ — золото для premium-tier
Premium	ravePremiumDeep	● #D97706	НОВЫЙ — тёмное золото для градиентов
Premium	ravePremiumLight	● #FCD34D	НОВЫЙ — светлое золото для glow
Успех	raveSuccess	● #10B981	Изумруд #00FF88 → приглушённый
Опасность	raveDanger	● #EF4444	Красный #FF3B5C → более чистый
Внимание	raveWarning	● #F59E0B	Янтарь (без изменений)
Secondary	raveSecondary	● #6366F1	НОВЫЙ — индиго
Фон	raveBackground	● #08080C	Чёрный #000000 → с фиолетовым подтоном
Surface	raveSurface	● #14141C	Карточный фон
Surface	raveSurfaceElevated	● #1F1F2A	Приподнятые карточки (featured)
Surface	raveSurfaceHover	● #2A2A38	Hover-состояние
Border	raveBorder	rgba(255,255,255,0.06)	Тонкая граница
Text	raveTextPrimary	● #FAFAFA	Чистый белый → чуть мягче
Text	raveTextSecondary	● #A1A1AA	Серый текст
Text	raveTextTertiary	● #71717A	Третий уровень

6.4 Градиенты

Градиент	Применение	Цвета
raveGradient	Primary CTA (Create Room, Confirm)	#A855F7 → #EC4899 (violet → magenta)
ravePremiumGradient	Subscribe button, premium cards	#FCD34D → #FBBF24 → #D97706 (gold)
ravePremiumRingGradient	Аватар premium-юзера (animated)	#FCD34D → #FBBF24 → #D97706 → #FCD34D
raveHostGradient	Аватар хоста в комнате	#EC4899 → #F43F5E (magenta → rose)
raveAdminGradient	Аватар админа (глобально)	#A855F7 → #6366F1 (violet → indigo)

raveBgGradient	Фоновый градиент	#08080C → #0F0A1A → #08080C
----------------	------------------	-----------------------------

6.5 Кнопки — 5 уровней

Каждый тип кнопки имеет чёткое назначение. Смешивание запрещено: например, нельзя использовать Primary-стиль для secondary-действий — это путает иерархию.

Тип	Применение	Фон	Текст	Тень/Glow	Pressed
Primary CTA	Create Room, Confirm, Send	raveGradient (violet→pink)	Белый bold	glowPrimary 16pt y=6	scale 0.96, glow ↓
Premium Subscribe	Upgrade, Restore Purchases	ravePremiumGradient (gold)	raveBackground (чёрный на золоте)	glowPremium 14pt y=5	scale 0.95, glow ↓
Secondary	Cancel, Back, Close	raveSurface	raveTextPrimary semibold	rgba(0,0,0,0.4) 10pt	scale 0.97
Ghost	Forgot password, Skip	transparent	ravePrimary medium	—	opacity 0.6, scale 0.98
Icon	Mute, Settings, Share	raveSurface circle	icon: raveTextSecondary	—	scale 0.9, spring

6.6 Аватарки — 4 уровня (КЛЮЧЕВОЕ)

Это самое важное решение дизайн-системы. Аватарка — это первый визуальный сигнал статуса. У free-юзера её обводки НЕ должно быть — премиум-эффект возникает через отсутствие, а не через добавление. Premium-юзер получает медленно вращающееся золотое кольцо (8s/оборот — не раздражает). Хост в контексте комнаты — статичная magenta-обводка. Админ — глобальная фиолетовая.

Роль	Обводка	Цвет	Анимация	Glow	Видимость
Free user	НЕТ обводки	—	—	—	Чистый circle, минимум визуального шума
Premium	2px animated gradient ring	ravePremiumRing Gradient (gold)	rotation 360° за 8s, linear, repeatForever	ravePremium.opacity(0.4) radius 6	Подписка видна всем, не навязчиво

Host (в комнате)	2px static gradient ring	raveHostGradient (magenta→rose)	—	raveAccent.opacity(0.35) radius 6	Только в контексте текущей комнаты
Admin / Moderator	2px static gradient ring	raveAdminGradient (violet→indigo)	—	ravePrimary.opacity(0.35) radius 6	Глобально во всём приложении

Правило приоритета (если у юзера несколько ролей одновременно):
Admin > Premium > Host > Free.
Например, premium-юзер, который сейчас хостит комнату, видит gold ring (premium приоритетнее host). Админ с premium-подпиской видит violet ring (admin приоритетнее premium).

6.7 Бейджи

Бейдж	Применение	Фон	Текст	Иконка	Анимация
LIVE	Активная комната	rgba(236,72,153,0.15)	raveAccent bold	pink dot, pulse 1s	pulse scale 1.0↔1.3
PREMIUM	Premium-юзер в чате	ravePremiumGradient	raveBackground heavy	crown.fill	—
HOST	Хост в списке участников	rgba(236,72,153,0.2)	raveAccent bold	star.fill	—
ADMIN	Админ в списке	rgba(168,85,247,0.15)	ravePrimary bold	shield.fill	—
MOD	Модератор комнаты	rgba(99,102,241,0.15)	raveSecondary bold	checkmark.shield	—

6.8 Карточки — 3 уровня

Уровень	Применение	Фон	Граница	Тень
Standard	Комнаты в списке, message bubbles	raveSurface (#14141C)	raveBorder 0.5px (rgba 0.06)	rgba(0,0,0,0.4) 10pt y=4
Premium Featured	Featured контент, premium-комнаты	raveSurfaceElevated (#1F1F2A)	gold gradient 0.5px (subtle)	glowPremium 8pt + black 10pt
Live / Active	Активные трансляции в HomeView	raveSurface + pink top stripe (2pt)	raveBorder 0.5px	rgba(0,0,0,0.4) 10pt, glowAccent on hover

6.9 Готовый код: Color+Theme.swift (полная замена)

□ RaveClone/RaveClone/Extensions/Color+Theme.swift

```
import SwiftUI
// MARK: - Cinema Violet – Premium Design System
// Заменяет "Pure Black × Ice Glow" (бирюзовый)
// Принципы:
// • База – глубокий чёрный с фиолетовым подтоном (#08080C)
// • Primary – фиолетовый (#A855F7), НЕ бирюзовый
// • Premium – золото (#FBBF24) для premium-tier сигналов
// • Accent – магента (#EC4899) для эмоциональных акцентов
extension Color {
    // — Accents —————
    /// Основной акцент – глубокий фиолетовый (СТА, активные элементы)
    static let ravePrimary = Color(hex: 0xA855F7)
    /// Магента – live, urgent, emotional
    static let raveAccent = Color(hex: 0xEC4899)
    /// Индиго – secondary
    static let raveSecondary = Color(hex: 0x6366F1)
    // — Premium Tier (gold) —————
    /// Премиум-золото – premium-tier, subscribe buttons
    static let ravePremium = Color(hex: 0xFBBF24)
    /// Тёмное золото – gradients
    static let ravePremiumDeep = Color(hex: 0xD97706)
    /// Светлое золото – glow, highlights
    static let ravePremiumLight = Color(hex: 0xFCD34D)
    // — Status —————
    /// Изумруд – online статус (приглушённый, не неон)
    static let raveSuccess = Color(hex: 0x10B981)
    /// Красный – danger, mute
    static let raveDanger = Color(hex: 0xEF4444)
    /// Янтарь – warning
    static let raveWarning = Color(hex: 0xF59E0B)
    // — Backgrounds —————
    /// Основной фон – глубокий чёрный с фиолетовым подтоном
    static let raveBackground = Color(hex: 0x08080C)
    /// Карточный фон
    static let raveSurface = Color(hex: 0x14141C)
    /// Приподнятые карточки (featured)
    static let raveSurfaceElevated = Color(hex: 0x1F1F2A)
    /// Hover-состояние
    static let raveSurfaceHover = Color(hex: 0x2A2A38)
    // — Borders —————
    static let raveBorder = Color.white.opacity(0.06)
    static let raveBorderHover = Color.white.opacity(0.12)
    // — Text —————
    static let raveTextPrimary = Color(hex: 0xFAFAFA)
    static let raveTextSecondary = Color(hex: 0xA1A1AA)
    static let raveTextTertiary = Color(hex: 0x71717A)
    // — Glass —————
    static let raveGlass = Color.white.opacity(0.05)
    // — Gradients —————
    /// Главный СТА: фиолетовый → магента
    static let raveGradient = LinearGradient(
        colors: [Color(hex: 0xA855F7), Color(hex: 0xEC4899)],
        startPoint: .topLeading, endPoint: .bottomTrailing
    )
}
```

```

)
/// Премиум-градиент: золотой перелив
static let ravePremiumGradient = LinearGradient(
    colors: [Color(hex: 0xFCD34D), Color(hex: 0xFBBF24), Color(hex: 0xD97706)],
    startPoint: .topLeading, endPoint: .bottomTrailing
)
/// Кольцо premium-аватара (animated rotation)
static let ravePremiumRingGradient = LinearGradient(
    colors: [Color(hex: 0xFCD34D), Color(hex: 0xFBBF24),
        Color(hex: 0xD97706), Color(hex: 0xFCD34D)],
    startPoint: .top, endPoint: .bottom
)
/// Хост-градиент (в контексте комнаты)
static let raveHostGradient = LinearGradient(
    colors: [Color(hex: 0xEC4899), Color(hex: 0xF43F5E)],
    startPoint: .topLeading, endPoint: .bottomTrailing
)
/// Админ-градиент (глобально)
static let raveAdminGradient = LinearGradient(
    colors: [Color(hex: 0xA855F7), Color(hex: 0x6366F1)],
    startPoint: .topLeading, endPoint: .bottomTrailing
)
/// Фоновый градиент — чёрный с фиолетовым
static let raveBgGradient = LinearGradient(
    colors: [Color(hex: 0x08080C), Color(hex: 0x0F0A1A), Color(hex: 0x08080C)],
    startPoint: .top, endPoint: .bottom
)
}
// MARK: - Hex init (без изменений)
extension Color {
    init(hex: UInt32, alpha: Double = 1.0) {
        let r = Double((hex >> 16) & 0xFF) / 255.0
        let g = Double((hex >> 8) & 0xFF) / 255.0
        let b = Double(hex & 0xFF) / 255.0
        self.init(.sRGB, red: r, green: g, blue: b, opacity: alpha)
    }
}
// MARK: - Glow Helpers
extension Color {
    var glowShadow: Color { self.opacity(0.45) }
}
extension View {
    func neonGlow(color: Color = .ravePrimary, radius: CGFloat = 16, y: CGFloat = 6) -> some View {
        self.shadow(color: color.glowShadow, radius: radius, x: 0, y: y)
    }
    func premiumGlow(radius: CGFloat = 12) -> some View {
        self.shadow(color: Color.ravePremium.opacity(0.5), radius: radius, x: 0, y: 4)
    }
}
}

```

6.10 Готовый код: PremiumAvatar.swift (новый файл)

□ RaveClone/RaveClone/Views/Components/PremiumAvatar.swift

```
import SwiftUI
```

```

// MARK: - Premium Avatar с role-based borders
//
// 4 уровня (приоритет: admin > premium > host > free):
// 1. Free user — БЕЗ обводки, чистый circle (минимум визуального шума)
// 2. Premium — анимированное золотое кольцо (8s/оборот, не раздражает)
// 3. Host — статичное magenta кольцо (только в текущей комнате)
// 4. Admin — статичное фиолетовое кольцо (глобально)
enum AvatarRole: Equatable {
    case free
    case premium
    case host
    case admin
    /// Приоритет для комбинирования ролей
    var priority: Int {
        switch self {
            case .free: return 0
            case .host: return 1
            case .premium: return 2
            case .admin: return 3
        }
    }
    /// Выбор доминирующей роли
    static func dominant(_ roles: [AvatarRole]) -> AvatarRole {
        roles.max(by: { $0.priority < $1.priority }) ?? .free
    }
}

struct PremiumAvatar: View {
    let imageURL: URL?
    let displayName: String
    let role: AvatarRole
    var size: CGFloat = 44
    @State private var animateRotation: Bool = false
    var body: some View {
        ZStack {
            avatarContent
                .frame(width: size, height: size)
                .clipShape(Circle())
            ringOverlay
                .frame(width: size + ringWidth, height: size + ringWidth)
        }
        .frame(width: size + ringWidth, height: size + ringWidth)
        .shadow(color: glowColor, radius: glowRadius)
        .onAppear { startAnimationIfNeeded() }
    }
    // MARK: - Avatar content (image or initials)
    @ViewBuilder
    private var avatarContent: some View {
        if let imageURL {
            AsyncImage(url: imageURL) { image in
                image.resizable().scaledToFill()
            } placeholder: {
                placeholderView
            }
        } else {
            placeholderView
        }
    }
}

```

```

    }
}
private var placeholderView: some View {
    ZStack {
        Circle().fill(Color.raveSurface)
        Text(String(displayName.prefix(1)).uppercased())
            .font(.system(size: size * 0.4, weight: .bold))
            .foregroundColor(.raveTextPrimary)
    }
}
// MARK: - Ring overlay (role-based) — КЛЮЧЕВАЯ логика
@ViewBuilder
private var ringOverlay: some View {
    switch role {
    case .free:
        // ◻ НЕТ обводки — чистый circle. Это и есть премиум-сигнал для остальных.
        EmptyView()
    case .premium:
        // ◻ Анимированное золотое кольцо
        Circle()
            .stroke(
                AngularGradient(
                    colors: [
                        Color(hex: 0xFCD34D),
                        Color(hex: 0xFBBF24),
                        Color(hex: 0xD97706),
                        Color(hex: 0xFCD34D),
                    ],
                    center: .center
                ),
                style: StrokeStyle(lineWidth: 2, lineCap: .round)
            )
            .rotationEffect(.degrees(animateRotation ? 360 : 0))
    case .host:
        // ◻ Статичное magenta кольцо — хост в текущей комнате
        Circle()
            .stroke(
                Color.raveHostGradient,
                style: StrokeStyle(lineWidth: 2)
            )
    case .admin:
        // ◻◻ Статичное фиолетовое кольцо — глобально
        Circle()
            .stroke(
                Color.raveAdminGradient,
                style: StrokeStyle(lineWidth: 2)
            )
    }
}
// MARK: - Geometry helpers
private var ringWidth: CGFloat {
    role == .free ? 0 : 4 // 2px stroke + 2px padding
}
private var glowColor: Color {
    switch role {

```

```

        case .free:      return .clear
        case .premium:   return .ravePremium.opacity(0.4)
        case .host:      return .raveAccent.opacity(0.35)
        case .admin:     return .ravePrimary.opacity(0.35)
    }
}

private var glowRadius: CGFloat {
    role == .free ? 0 : 6
}

private func startAnimationIfNeeded() {
    guard role == .premium, !animateRotation else { return }
    withAnimation(.linear(duration: 8).repeatForever(autoreverses: false)) {
        animateRotation = true
    }
}
}

// MARK: - Usage examples
/*
// Free user в списке участников:
PremiumAvatar(imageURL: user.avatarURL, displayName: user.name, role: .free, size: 44)
// Premium-юзер:
PremiumAvatar(imageURL: user.avatarURL, displayName: user.name, role: .premium, size: 44)
// Хост в RoomView:
PremiumAvatar(imageURL: host.avatarURL, displayName: host.name,
               role: .dominant([.premium, .host]), size: 56)
               // → выберет .premium (приоритет выше)
// Админ в AdminPanelView:
PremiumAvatar(imageURL: admin.avatarURL, displayName: admin.name, role: .admin, size: 44)
*/

```

6.11 Готовый код: PremiumButtonStyle.swift (5 стилей)

▢ RaveClone/RaveClone/Views/Components/PremiumButtonStyle.swift (заменяет существующий)

```

import SwiftUI

// MARK: - 1. Primary CTA (violet → magenta gradient)
// Применение: Create Room, Confirm, Send, Join
struct PremiumButtonStyle: ButtonStyle {
    var gradient: LinearGradient = Color.raveGradient
    var glowColor: Color = .ravePrimary

    func makeBody(configuration: Configuration) -> some View {
        configuration.label
            .font(.system(size: 16, weight: .bold, design: .rounded))
            .foregroundColor(.white)
            .scaleEffect(configuration.isPressed ? 0.96 : 1.0)
            .animation(.spring(response: 0.3, dampingFraction: 0.7),
                      value: configuration.isPressed)
            .shadow(
                color: glowColor.opacity(configuration.isPressed ? 0.2 : 0.45),
                radius: configuration.isPressed ? 6 : 16,
                y: 6
            )
    }
}

// MARK: - 2. Premium Subscribe (gold gradient)

```

```

// Применение: Upgrade to Premium, Restore Purchases
struct PremiumSubscribeButtonStyle: ButtonStyle {
    func makeBody(configuration: Configuration) -> some View {
        configuration.label
            .font(.system(size: 16, weight: .heavy, design: .rounded))
            .foregroundColor(.raveBackground) // чёрный на золоте – макс контраст
            .scaleEffect(configuration.isPressed ? 0.95 : 1.0)
            .animation(.spring(response: 0.3, dampingFraction: 0.6),
                value: configuration.isPressed)
            .shadow(
                color: .ravePremium.opacity(configuration.isPressed ? 0.3 : 0.55),
                radius: configuration.isPressed ? 4 : 14,
                y: 5
            )
    }
}

// MARK: - 3. Secondary (surface + subtle border)
// Применение: Cancel, Back, Close
struct SecondaryButtonStyle: ButtonStyle {
    func makeBody(configuration: Configuration) -> some View {
        configuration.label
            .font(.system(size: 15, weight: .semibold))
            .foregroundColor(.raveTextPrimary)
            .scaleEffect(configuration.isPressed ? 0.97 : 1.0)
            .animation(.spring(response: 0.3, dampingFraction: 0.7),
                value: configuration.isPressed)
    }
}

// MARK: - 4. Ghost (text only)
// Применение: Forgot password, Skip, View all
struct GhostButtonStyle: ButtonStyle {
    var color: Color = .ravePrimary
    func makeBody(configuration: Configuration) -> some View {
        configuration.label
            .font(.system(size: 14, weight: .medium))
            .foregroundColor(color)
            .opacity(configuration.isPressed ? 0.6 : 1.0)
            .scaleEffect(configuration.isPressed ? 0.98 : 1.0)
    }
}

// MARK: - 5. Icon (mute, settings, share)
// Применение: круглые icon-кнопки
struct IconButtonStyle: ButtonStyle {
    var isActive: Bool = false
    var activeColor: Color = .raveAccent
    func makeBody(configuration: Configuration) -> some View {
        configuration.label
            .font(.system(size: 17, weight: .semibold))
            .foregroundColor(isActive ? activeColor : .raveTextSecondary)
            .scaleEffect(configuration.isPressed ? 0.9 : 1.0)
            .animation(.spring(response: 0.25, dampingFraction: 0.6),
                value: configuration.isPressed)
    }
}

// MARK: - Modifier helpers

```

```

extension View {
    func primaryButton() -> some View {
        self
            .background(Color.raveGradient)
            .clipShape(RoundedRectangle(cornerRadius: 16))
            .buttonStyle(PremiumButtonStyle())
    }
    func premiumSubscribeButton() -> some View {
        self
            .background(Color.ravePremiumGradient)
            .clipShape(RoundedRectangle(cornerRadius: 16))
            .buttonStyle(PremiumSubscribeButtonStyle())
    }
    func secondaryButton() -> some View {
        self
            .background(Color.raveSurface)
            .overlay(
                RoundedRectangle(cornerRadius: 14)
                    .stroke(Color.raveBorder, lineWidth: 0.5)
            )
            .clipShape(RoundedRectangle(cornerRadius: 14))
            .buttonStyle(SecondaryButtonStyle())
    }
    func ghostButton(color: Color = .ravePrimary) -> some View {
        self.buttonStyle(GhostButtonStyle(color: color))
    }
    func iconButton(isActive: Bool = false, activeColor: Color = .raveAccent) -> some View {
        self
            .frame(width: 44, height: 44)
            .background(Color.raveSurface)
            .overlay(Circle().stroke(Color.raveBorder, lineWidth: 0.5))
            .clipShape(Circle())
            .buttonStyle(IconButtonStyle(isActive: isActive, activeColor: activeColor))
    }
}

```

6.12 Готовый код: AnimatedGradientBackground.swift (обновлённый)

Объединяет фикс из раздела 02 (TimelineView вместо withAnimation) с новой палитрой «Cinema Violet» — фоновые орбы теперь фиолетовые, магента и золотые (вместо бирюзовых).

▢ RaveClone/RaveClone/Views/Components/AnimatedGradientBackground.swift (полная замена)

```

import SwiftUI
// MARK: - Cinema Violet Ambient Background
// Заменяет бирюзовый "Ice Glow" на фиолетово-золотой premium-фон.
// TimelineView — надёжная анимация (не зависит от lifecycle).
struct AnimatedGradientBackground: View {
    var hasActiveRooms: Bool = true
    var body: some View {
        ZStack {
            // База — глубокий чёрный с фиолетовым подтоном
            Color.raveBackground

```

```

        .ignoresSafeArea()
    // Плавающие glow-orbs (violet, magenta, gold)
    TimelineView(.animation) { context in
        let t = context.date.timeIntervalSinceReferenceDate
        ZStack {
            glowOrb(
                color: .ravePrimary,          // фиолетовый
                t: t, size: 360, blur: 50,
                xAmp: 60, yAmp: 180,
                xPhase: 0.0, yPhase: 0.3,
                opacity: hasActiveRooms ? 0.40 : 0.25
            )
            glowOrb(
                color: .raveAccent,           // магента
                t: t, size: 300, blur: 60,
                xAmp: 100, yAmp: 160,
                xPhase: 1.2, yPhase: 0.7,
                opacity: hasActiveRooms ? 0.32 : 0.20
            )
            glowOrb(
                color: .ravePremium,          // золото (premium hint)
                t: t, size: 240, blur: 70,
                xAmp: 80, yAmp: 40,
                xPhase: 2.4, yPhase: 1.1,
                opacity: hasActiveRooms ? 0.20 : 0.15
            )
        }
        .ignoresSafeArea()
    }
}

private func glowOrb(
    color: Color, t: Double, size: CGFloat, blur: CGFloat,
    xAmp: CGFloat, yAmp: CGFloat,
    xPhase: Double, yPhase: Double,
    opacity: Double
) -> some View {
    Circle()
        .fill(color.opacity(opacity))
        .frame(width: size, height: size)
        .blur(radius: blur)
        .offset(
            x: CGFloat(sin(t * 0.1 + xPhase)) * xAmp,
            y: CGFloat(cos(t * 0.07 + yPhase)) * yAmp
        )
        .blendMode(.screen) // свечение поверх чёрного
    }
}

// MARK: - GlassCard (обновлён под новую палитру)
struct GlassCard: ViewModifier {
    var cornerRadius: CGFloat = 18
    var opacity: Double = 0.05
    func body(content: Content) -> some View {
        content
            .background(

```

```

        ZStack {
            RoundedRectangle(cornerRadius: cornerRadius)
                .fill(Color.white.opacity(opacity))
            RoundedRectangle(cornerRadius: cornerRadius)
                .fill(
                    LinearGradient(
                        colors: [
                            Color.white.opacity(0.10),
                            Color.clear
                        ],
                        startPoint: .top, endPoint: .center
                    )
                )
        }
    )
    .background(.ultraThinMaterial)
    .clipShape(RoundedRectangle(cornerRadius: cornerRadius))
    .overlay(
        RoundedRectangle(cornerRadius: cornerRadius)
            .stroke(
                LinearGradient(
                    colors: [
                        Color.white.opacity(0.18),
                        Color.white.opacity(0.04),
                    ],
                    startPoint: .topLeading,
                    endPoint: .bottomTrailing
                ),
                lineWidth: 0.5
            )
    )
    .shadow(color: Color.black.opacity(0.4), radius: 10, y: 4)
}

extension View {
    func glassCard(cornerRadius: CGFloat = 18, opacity: Double = 0.05) -> some View {
        modifier(GlassCard(cornerRadius: cornerRadius, opacity: opacity))
    }
}

// MARK: - Badge components (LIVE, PREMIUM, HOST, ADMIN)
struct LiveBadge: View {
    @State private var pulse = false
    var body: some View {
        HStack(spacing: 4) {
            Circle()
                .fill(Color.raveAccent)
                .frame(width: 6, height: 6)
                .scaleEffect(pulse ? 1.3 : 1.0)
            Text("LIVE")
                .font(.system(size: 10, weight: .heavy))
                .foregroundColor(.raveAccent)
        }
        .padding(.horizontal, 8)
        .padding(.vertical, 5)
        .background(Color.raveAccent.opacity(0.15))
    }
}

```

```

        .clipShape(Capsule())
        .overlay(Capsule().stroke(Color.raveAccent.opacity(0.5), lineWidth: 0.5))
        .shadow(color: .raveAccent.opacity(0.4), radius: pulse ? 8 : 3)
        .onAppear {
            withAnimation(.easeInOut(duration: 1.0).repeatForever(autoreverses: true)) {
                pulse = true
            }
        }
    }
}

struct PremiumBadge: View {
    var body: some View {
        HStack(spacing: 3) {
            Image(systemName: "crown.fill")
                .font(.system(size: 9))
            Text("PREMIUM")
                .font(.system(size: 9, weight: .heavy))
        }
        .foregroundColor(.raveBackground) // чёрный на золоте
        .padding(.horizontal, 6)
        .padding(.vertical, 3)
        .background(Color.ravePremiumGradient)
        .clipShape(Capsule())
        .shadow(color: .ravePremium.opacity(0.4), radius: 4)
    }
}

struct HostBadge: View {
    var body: some View {
        HStack(spacing: 3) {
            Image(systemName: "star.fill")
                .font(.system(size: 9))
            Text("HOST")
                .font(.system(size: 9, weight: .heavy))
        }
        .foregroundColor(.raveAccent)
        .padding(.horizontal, 6)
        .padding(.vertical, 3)
        .background(Color.raveAccent.opacity(0.2))
        .clipShape(Capsule())
        .overlay(Capsule().stroke(Color.raveAccent.opacity(0.5), lineWidth: 0.5))
    }
}

struct AdminBadge: View {
    var body: some View {
        HStack(spacing: 3) {
            Image(systemName: "shield.fill")
                .font(.system(size: 9))
            Text("ADMIN")
                .font(.system(size: 9, weight: .heavy))
        }
        .foregroundColor(.ravePrimary)
        .padding(.horizontal, 6)
        .padding(.vertical, 3)
        .background(Color.ravePrimary.opacity(0.15))
        .clipShape(Capsule())
    }
}

```

```
        .overlay(Capsule().stroke(Color.ravePrimary.opacity(0.5), lineWidth: 0.5))
    }
}
```

6.13 План миграции

Миграция состоит из 4 шагов. Каждый шаг — отдельный коммит. Прогресс виден визуально после каждого.

Шаг	Действие	Файлы	Результат
1	Заменить Color+Theme.swift целиком	RaveClone/RaveClone/Extensions/Color+Theme.swift	Все ссылки на ravePrimary/raveAccent автоматически получают новые цвета. Бирюзовый исчезает.
2	Добавить PremiumAvatar.swift + Badge components	RaveClone/RaveClone/Views/Components/PremiumAvatar.swift (новый) AnimatedGradientBackground.swift (обновить секцию badges)	Доступны 4 уровня аватарок и 4 бейджа. Старый код продолжает работать.
3	Обновить AnimatedGradientBackground.swift (фон + TimelineView fix)	RaveClone/RaveClone/Views/Components/AnimatedGradientBackground.swift	Фон становится фиолетово-золотым. Заодно фиксится баг C1 раздела 02 (анимация).
4	Заменить старые аватарки на PremiumAvatar во views	Views/Home/HomeView.swift Views/Room/ParticipantListView.swift Views/Room/RoomChatView.swift Views/Profile/ProfileView.swift Views/Friends/FriendsView.swift Views/Admin/AdminPanelView.swift	Аватарки получают role-based обводки. Premium-юзеры видят gold ring, free — без обводки.
5	Обновить кнопки: заменить .buttonStyle(PremiumButtonStyle()) на специфичные стили	Views/Home/HomeView.swift Views/Auth/LoginView.swift Views/Room/RoomView.swift Views/Premium/PaywallView.swift	Premium Subscribe кнопки получают gold gradient. Secondary кнопки — surface+border.
6	Добавить PremiumBadge в chat messages и participant list	Views/Room/RoomChatView.swift Views/Room/ParticipantListView.swift	Premium-юзеры видны в чате по gold-бейджу рядом с именем.

6.14 Правила дизайна (для всех будущих экранов)

- Бирюзовый запрещён. Никаких #00C2FF, #00FFCC, #00FF88. Если видите — заменить на ravePrimary (violet) или raveSuccess (приглушённый).
- Золото = только premium. ravePremium (#FBBF24) используется ТОЛЬКО для premium-tier сигналов: avatar ring, subscribe button, premium badge, premium-featured карточки. Не использовать для обычных СТА.

- Free-юзер — без визуального шума. У бесплатного юзера на аватаре нет обводки, нет бейджа, нет glow. Это и есть мотивация апгрейтить — разница видна сразу.
- Один primary CTA на экран. Если на экране две кнопки с violet→magenta gradient — это ошибка. Secondary должна быть SecondaryButtonStyle (surface + border).
- Pressed state — обязателен. Все интерактивные элементы имеют scaleEffect на press (0.95-0.97). Без pressed-state кнопка ощущается мёртвой.
- Backgrounds — глубокий чёрный. Не серый, не фиолетовый. #08080C. Это «кинозал».
- Скругления — consistent. Cards = 16pt, buttons = 14-16pt, badges = capsule. Не смешивать.
- Тени — цветные, не чёрные. Primary кнопка — violet glow. Premium — gold glow. Обычные карточки — чёрная тень. Цветная тень = premium-ощущение.
- Анимации — медленные и плавные. Premium ring вращается 8s/оборот (не 2s). Pulse LIVE — 1s. Никаких резких bounce-эффектов на premium-элементах.
- Граница 0.5px. Тонкие границы выглядят дороже, чем 1-2px. Использовать raveBorder (rgba 0.06) на всех стандартных карточках.

6.15 Чек-лист для App Store Review

После миграции на Cinema Violet убедитесь, что:

- ✓ Premium-подписка визуально отличается от free (gold ring + badge + subscribe button)
- ✓ Все premium-фичи (4K, no-ads, 50 participants) доступны только при активной подписке
- ✓ Subscribe button использует gold gradient, не violet (чёткое визуальное разделение)
- ✓ Restore Purchases работает (см. баг iOS M5 —□□ить в этапе 4 дорожной карты)
- ✓ Free-юзеры не видят gold-элементов нигде (иначе вводит в заблуждение)
- ✓ Premium ring не анимируется быстрее 8s/оборот (иначе раздражает — App Store reject за анимацию)
- ✓ Все цвета соответствуют контрастности WCAG AA (минимум 4.5:1 для текста)
- ✓ Dark mode единственный (light mode не поддерживается — приложение для тёмных помещений)

07 **Дорожная карта исправлений**

Рекомендуемый порядок фиксов. Каждый этап — independently shippable: можно релизить после каждого.

Этап	Срок	Что	Баги	Результат
1. Realtime unlock	1-2 дня	iOS C1 (WebSocket lifecycle) + Backend B1+B2 (user.id + admin auth)	iOS C1, B1, B2	Чат, sync, signaling начинают работать. Admin panel доступен.
2. Auth security	2-3 дня	JWT в Keychain (iOS C2), SecureStore (RN R2). Удалить demo-bypass (RN R4+R5). Реализовать refresh (iOS C3).	iOS C2,C3; RN R2,R3,R4,R5	Auth модель trustworthy. App Store pre-review passed.
3. Service DI	2-3 дня	Внедрить общий authenticated APIClient в DM/Friends/Admin (iOS C4+C5+C6). Прокинуть currentUserId и hostIsPremium (iOS C7+C8).	iOS C4-C8	DM, друзья, админка работают. Сync-движок запускается от хоста.
4. IAP & premium	3-4 дня	Удалить setPremium (iOS C9). Реализовать серверный /iap/verify. Починить restorePurchases (iOS M5). Удалить B5 (free premium bypass).	iOS C9, M5; B5	Premium-модель безопасна. App Store Guideline 3.1.1 satisfied.
5. iOS infra	3-4 дня	Один AVPlayer (iOS H3), удалить утечки таймеров (H4,H5,H8), unbounded messages (H13), Sendable APIClient (H10).	iOS H3,H4,H5,H8,H10,H13	Стабильный iOS, нет memory leaks, sync работает визуально.
6. RN playback	3-5 дней	WebViewPlayer правильный URL (R11), runtime JS injection (R12), YouTube через WebView (R10).	RN R10,R11,R12	Видео проигрывается на RN, sync работает.
7. RN WebRTC	3-4 дня	Mute button (R15), webrtc_leave с roomId (R16), TURN server (R18), не мьютить на transient disconnect (R14).	RN R14,R15,R16,R18	Voice chat стабилен.
8. Backend security	2-3 дня	SSRF protection (B10), yt-dlp arg injection (B12), chat impersonation (B3), ban enforcement (B4).	B3,B4,B10,B12	Backend hardened. Готов к public exposure.
9. Polish	1-2 недели	Все Medium и Low баги. Localizations, dead code cleanup, RN strict mode,	Все M/L	Production-ready v1.0.

Финальные заметки для GLM-5.2

Этот отчёт — снимок состояния кодовой базы RaveClone на 3 июля 2026 года. Все 155 багов подтверждены чтением исходного кода. Bug IDs стабильны: C1-H14-M1-L16 (iOS), B1-B34 (Backend), R1-R61 (React Native). При обсуждении фиксов ссылайтесь на ID — это однозначно идентифицирует баг.

Приоритезация для v1.0 MVP: этапы 1-4 дорожной карты обязательны до релиза. Этапы 5-9 можно делать постепенно, но этап 5 (iOS infra) желателен до App Store submission — иначе review-команда Apple точно найдёт утечки памяти и crash-баги.

Спец-задача с фоном (раздел 02) не блокирует релиз, но влияет на user experience. Рекомендуется fix в этапе 9 (Polish), либо сразу после этапа 1, поскольку fix простой (один файл, ~50 строк).

Дальнейшие шаги: после фикса этапов 1-4 провести повторный audit только критических подсистем (auth, sync, IAP) — они самые рискованные. Использовать этот отчёт как baseline для регрессии.